



Supply Chain Management

Report created by
Muthuraman Ramasamy

March 2026

Table of Contents:

List of Figures	4
Understanding the Business Problem	7
Context	7
Objective	7
Data Description	7
Data Overview	8
Checking the data types of the columns for the dataset	8
Checking the shape of the dataset	8
Displaying the first and last few rows of the dataset	8
Checking for missing values	9
Statistical summary of the dataset	10
Checking for duplicate values	11
Exploratory Data Analysis (EDA)	12
Univariate analysis	12
Bivariate analysis	20
Overall Summary of EDA Observations:	33
Check Missing Values	34
Identify and Visualize Outliers	35
Summary of Outlier Handling Steps:	36
Feature Engineering	37
Handle Class Imbalance (if needed)	37
Feature Scaling	38
Data Preparation for Modeling	38
Handle Data Leakage	38
Data Analysis Key Findings	39
Model Building – Baseline Model	40
Decide Metric of Choice with Rationale	40
Build a Baseline Linear Regression Model	40
Baseline Model Performance	41
Model Building - Advanced Models	42
Decision Tree Model	42
Bagging Regressor	42
Random Forest Model	43
AdaBoost Regressor	43
Gradient Boosting Regressor	44
XGBoost Regressor	44

Model Performance Improvement using Hyperparameter Tuning	45
Hyperparameter Tuning - Decision Tree.....	45
Hyperparameter Tuning - Bagging Regressor	45
Hyperparameter Tuning - Random Forest	46
Hyperparameter Tuning - AdaBoost Regressor	46
Hyperparameter Tuning - Gradient Boosting Regressor	47
Hyperparameter Tuning - XGBoost Regressor	47
Stacking Model.....	48
Model Comparison and Final Model Selection	49
Training Performance Comparison	49
Testing performance comparison	49
Best Model Selection	50
Actionable Insights & Recommendations.....	50
Key Insights from Analysis.....	50
Actionable Business Recommendations.....	51
Business Impact.....	52

List of Figures

Figure 1: List of Data Set Variables	8
Figure 2: First few rows from the dataset	8
Figure 3: Last few rows from the dataset	9
Figure 4: Checking for missing values.....	9
Figure 5: Missing Value percentages	9
Figure 6: Statistical Summary for Numerical Variables	10
Figure 7: Statistical Summary for Categorical Variables	10
Figure 8: Finding duplicates	11
Figure 9: Observations on product_wg_ton	12
Figure 10: Observations on num_refill_req_l3m	12
Figure 11: Observations on retail_shop_num	13
Figure 12: Observations on dist_from_hub	13
Figure 13: Observations on workers_num.....	14
Figure 14: Observations on wh_est_year.....	14
Figure 15: Observations on Location_type.....	15
Figure 16: Observations on WH_capacity_size	15
Figure 17: Observations on zone	16
Figure 18: Observations on wh_owner_type.....	16
Figure 19: Observations on flood_impacted.....	17
Figure 20: Observations on flood_proof	17
Figure 21: Observations on electric_supply	18
Figure 22: Observations on temp_reg_mach	18
Figure 23: Observations on approved_wh_govt_certificate	19
Figure 24: Observations on retail_shop_num vs product_wg_ton by zone	20
Figure 25: Observations on distributor_num vs product_wg_ton by zone	20
Figure 26: Observations on dist_from_hub vs product_wg_ton by zone	21
Figure 27: Observations on workers_num vs product_wg_ton by zone	21
Figure 28: Observations on num_refill_req_l3m vs product_wg_ton by zone	22
Figure 29: Observations on retail_shop_num vs product_wg_ton (Regression Plot)	22
Figure 30: Observations on dist_from_hub vs product_wg_ton (Regression Plot).....	23
Figure 31: Observations on workers_num vs product_wg_ton (Regression Plot)	23
Figure 32: Observations on Correlation Heatmap - Continuous Variables	24
Figure 33: Observations on zone vs flood_impacted (Stacked Bar Chart)	25
Figure 34: Observations on zone vs electric_supply (Stacked Bar Chart).....	25
Figure 35: Observations on zone vs wh_owner_type (Stacked Bar Chart)	26
Figure 36: Observations on zone vs product_wg_ton (Box Plot)	26
Figure 37: Observations on Location_type vs product_wg_ton (Box Plot)	27
Figure 38: Observations on WH_regional_zone vs product_wg_ton (Box Plot)	27
Figure 39: Observations on wh_owner_type vs product_wg_ton (Box Plot).....	28
Figure 40: Observations on approved_wh_govt_certificate vs product_wg_ton (Box Plot).....	28
Figure 41: Observations on flood_impacted vs product_wg_ton (Box Plot)	29

Figure 42: Observations on flood_proof vs product_wg_ton (Box Plot).....	29
Figure 43: Observations on Average Product Weight by Zone and Regional (Heatmap)	30
Figure 44: Observations on distance_bin vs product_wg_ton (Bar Plot)	30
Figure 45: Observations on WH_capacity_size vs product_wg_ton (Box Plot)	31
Figure 46: Observations on Competitor_in_mkt vs product_wg_ton (Box Plot).....	31
Figure 47: Observations on transport_issue_l1y vs product_wg_ton (Box Plot)	32
Figure 48: Observations on storage_issue_reported_l3m vs product_wg_ton (Box Plot).....	32
Figure 49: Missing Values percentages	34
Figure 50: Outliers in 'product_wg_ton' treated using capping.....	35
Figure 51: Outliers in 'retail_shop_num' treated using capping.....	35
Figure 52: Outliers in 'workers_num' treated using capping	36
Figure 53: First Few Rows after Ordinal Feature Creation.....	37
Figure 54: First 5 rows of X_train with re-scaled features	39
Figure 55: Linear Regression	40
Figure 56: Training Data - Linear Regression	40
Figure 57: Test 'Data - Linear Regression.....	40
Figure 58: Score - Linear Regression.....	41
Figure 59: Decision Tree Regressor.....	42
Figure 60: Performance on Training Set - Decision Tree Regressor	42
Figure 61: Performance on Test Set - Decision Tree Regressor	42
Figure 62: Bagging Regressor.....	42
Figure 63: Performance on Training Set - Bagging Regressor	42
Figure 64: Performance on Test Set - Bagging Regressor	43
Figure 65: Random Forest Regressor	43
Figure 66: Performance on Training Set - RF Regressor.....	43
Figure 67: Performance on Test Set - RF Regressor.....	43
Figure 68: AdaBoost Regressor	43
Figure 69: Performance on Training Set - AdaBoost Regressor.....	43
Figure 70: Performance on Test Set - AdaBoost Regressor.....	43
Figure 71: Gradient Boosting Regressor	44
Figure 72: Performance on Training Set - GB Regressor	44
Figure 73: Performance on Test Set - GB Regressor	44
Figure 74: XGBoost Regressor	44
Figure 75: Performance on Training Set - XGB Regressor.....	44
Figure 76: Performance on Test Set - XGB Regressor.....	44
Figure 77: Hyperparameter Tuning - Decision Tree	45
Figure 78: Tuned Decision Tree of Traing Set	45
Figure 79: Tuned Decision Tree of Test Set	45
Figure 80: Hyperparameter Tuning - Bagging Regressor	45
Figure 81: Tuned Bagging Regressor of Training Set.....	45
Figure 82: Tuned Bagging Regressor of Test Set.....	45
Figure 83: Hyperparameter Tuning - Random Forest	46
Figure 84: Tuned Random Forest of Training Set.....	46

Figure 85: Tuned Random Forest of Test Set.....	46
Figure 86: Hyperparameter Tuning - AdaBoost Regressor.....	46
Figure 87: Tuned AdaBoost Regressor of Testing Set.....	46
Figure 88: Tuned AdaBoost Regressor of Test Set	46
Figure 89: Hyperparameter Tuning - Gradient Boosting Regressor	47
Figure 90: Tuned Gradient Boosting Regressor of Training Set	47
Figure 91: Tuned Gradient Boosting Regressor of Test Set	47
Figure 92: Hyperparameter Tuning - XGBoost Regressor	47
Figure 93: Tuned XGBoost Regressor of Training Set.....	47
Figure 94: Tuned XGBoost Regressor of Test Set.....	47
Figure 95: Stacking Model.....	48
Figure 96: Stacking Model - Training Set	48
Figure 97: Stacking Model - Test Set	48
Figure 98: Training Performance Comparison	49
Figure 99: Testing performance comparison.....	49

Understanding the Business Problem

Context

A fast-moving consumer goods (FMCG) company that entered the instant noodles market two years ago is facing challenges in balancing demand and supply across its warehouses nationwide. In some regions, demand far exceeds supply, while in others, excess inventory is leading to wastage and higher costs. This mismatch results in significant inventory losses and missed sales opportunities. To remain competitive and profitable in the highly dynamic FMCG sector, the company needs a data-driven approach to optimize its supply chain and align production with regional demand patterns.

Objective

As a data scientist, you are tasked with building a predictive model using historical sales and supply data to determine the optimum quantity of product to ship to each warehouse. Additionally, analyzing demand patterns across different regions will help management design targeted advertisement campaigns. Solving this problem will enable the company to minimize inventory costs, reduce stockouts, and drive higher sales through better demand forecasting and efficient supply chain management.

The company is facing demand-supply imbalance across warehouses. Some regions are experiencing stockouts (lost sales), while others have overstocking (increased holding cost & wastage). The core business challenge is optimizing shipment quantities to each warehouse based on predicted regional demand, so that:

- Inventory cost is minimized
- Stockouts are reduced
- Sales opportunities are maximized

Data Description

The dataset contains information about the historical sales patterns. The detailed data dictionary is given below:

- Ware_house_ID: Product warehouse ID
- WH_Manager_ID: Employee ID of warehouse manager
- Location_type: Location of the warehouse, like in a city or a village
- WH_capacity_size: Storage capacity size of the warehouse
- zone: Zone of the warehouse
- WH_regional_zone: Regional zone of the warehouse under each zone
- num_refill_req_l3m: Number of times refilling has been done in the last 3 months
- transport_issue_l1y: Any transport issue, like an accident or goods stolen, reported in the last year
- Competitor_in_mkt: Number of instant noodles competitors in the market
- retail_shop_num: Number of retail shops that sell the product under the warehouse area
- wh_owner_type: Company owns the warehouse, or they have rented the warehouse
- distributor_num: Number of distributors working between the warehouse and retail shops
- flood_impacted: Warehouse is in the flood-impacted area indicator
- flood_proof: Warehouse is flood-proof. Like storage is at some height, not directly on the ground
- electric_supply: The Warehouse has an electric backup, like a generator, so they can run the warehouse in load shedding
- dist_from_hub: Distance between the warehouse to the production hub in Kms
- workers_num: Number of workers working in the warehouse
- wh_est_year: Warehouse established year
- storage_issue_reported_l3m: Warehouse reported a storage issue to the corporate office in the last 3 months. Like rats, fungus due to moisture, etc.
- temp_reg_mach: Warehouse has a temperature regulating machine indicator

- `approved_wh_govt_certificate`: What kind of standard certificate has been issued to the warehouse from the government regulatory body
- `wh_breakdown_l3m`: Number of times the warehouse faced a breakdown in the last 3 months. Like a strike by workers, a flood, or an electrical failure
- `govt_check_l3m`: Number of times government officers have visited the warehouse to check the quality and expiry of stored food in the last 3 months
- `product_wg_ton`: Product has been shipped in the last 3 months. Weight is in tons

Data Overview

The initial steps to get an overview of any dataset is to:

- Observe the first few rows of the dataset, to check whether the dataset has been loaded properly or not
- Get information about the number of rows and columns in the dataset
- Find out the data types of the columns to ensure that data is stored in the preferred format and the value of each property is as expected.
- Check the statistical summary of the dataset to get an overview of the numerical columns of the data

Checking the data types of the columns for the dataset

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25000 entries, 0 to 24999
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Ware_house_ID                        25000 non-null  object
1   WH_Manager_ID                       25000 non-null  object
2   Location_type                       25000 non-null  object
3   WH_capacity_size                    25000 non-null  object
4   zone                                25000 non-null  object
5   WH_regional_zone                    25000 non-null  object
6   num_refill_req_l3m                 25000 non-null  int64
7   transport_issue_l1y                25000 non-null  int64
8   Competitor_in_mkt                  25000 non-null  int64
9   retail_shop_num                     25000 non-null  int64
10  wh_owner_type                       25000 non-null  object
11  distributor_num                     25000 non-null  int64
12  flood_impacted                      25000 non-null  int64
13  flood_proof                         25000 non-null  int64
14  electric_supply                     25000 non-null  int64
15  dist_from_hub                       25000 non-null  int64
16  workers_num                         24010 non-null  float64
17  wh_est_year                         13119 non-null  float64
18  storage_issue_reported_l3m          25000 non-null  int64
19  temp_reg_mach                       25000 non-null  int64
20  approved_wh_govt_certificate        24092 non-null  object
21  wh_breakdown_l3m                   25000 non-null  int64
22  govt_check_l3m                     25000 non-null  int64
23  product_wg_ton                     25000 non-null  int64
dtypes: float64(2), int64(14), object(8)
memory usage: 4.6+ MB
```

Figure 1: List of Data Set Variables

- There are 16 numeric (float and int type) and 8 string (object type) columns in the data
- The target variable is the "product_wg_ton", which is of int type

Checking the shape of the dataset

The dataset contains 24 variables and 25,000 rows

Displaying the first and last few rows of the dataset

	Ware_house_ID	WH_Manager_ID	Location_type	WH_capacity_size	zone	WH_regional_zone	num_refill_req_l3m	transport_issue_l1y	Competitor_in_mkt	retail_shop_num	...	electric_supply	dist_from_hub
0	WH_100000	EID_50000	Urban	Small	West	Zone 6	3	1	2	4651	...	1	91
1	WH_100001	EID_50001	Rural	Large	North	Zone 5	0	0	4	6217	...	1	210
2	WH_100002	EID_50002	Rural	Mid	South	Zone 2	1	0	4	4306	...	0	161
3	WH_100003	EID_50003	Rural	Mid	North	Zone 3	7	4	2	6000	...	0	103
4	WH_100004	EID_50004	Rural	Large	North	Zone 5	3	1	2	4740	...	1	112

Figure 2: First few rows from the dataset

	Ware_house_ID	WH_Manager_ID	Location_type	WH_capacity_size	zone	WH_regional_zone	num_refill_req_l3m	transport_issue_l1y	Competitor_in_mkt	retail_shop_num	...	electric_supply	dist_from_hul
24995	WH_124995	EID_74995	Rural	Small	North	Zone 1	3	0	4	5390	...	1	14
24996	WH_124996	EID_74996	Rural	Mid	West	Zone 2	6	0	4	4490	...	1	13
24997	WH_124997	EID_74997	Urban	Large	South	Zone 5	7	0	2	5403	...	1	14
24998	WH_124998	EID_74998	Rural	Small	North	Zone 1	1	0	2	10562	...	1	6
24999	WH_124999	EID_74999	Rural	Mid	West	Zone 4	8	2	4	5664	...	1	23

5 rows × 24 columns

Figure 3: Last few rows from the dataset

Checking for missing values

	0
Ware_house_ID	0
WH_Manager_ID	0
Location_type	0
WH_capacity_size	0
zone	0
WH_regional_zone	0
num_refill_req_l3m	0
transport_issue_l1y	0
Competitor_in_mkt	0
retail_shop_num	0
wh_owner_type	0
distributor_num	0
flood_impacted	0
flood_proof	0
electric_supply	0
dist_from_hub	0
workers_num	990
wh_est_year	11881
storage_issue_reported_l3m	0
temp_reg_mach	0
approved_wh_govt_certificate	908
wh_breakdown_l3m	0
govt_check_l3m	0
product_wg_ton	0

dtype: int64

Figure 4: Checking for missing values

There are missing values in 3 columns:

- workers_num = 990, which is almost 4%
- wh_est_year = 11881, which is almost 48%
- approved_wh_govt_certificate = 908, which is almost 4%

	Missing Values	Percentage	Data Type
wh_est_year	11881	47.52	float64
workers_num	990	3.96	float64
approved_wh_govt_certificate	908	3.63	object

Figure 5: Missing Value percentages

Statistical summary of the dataset

	count	mean	std	min	25%	50%	75%	max
num_refill_req_l3m	25000.00	4.09	2.61	0.00	2.00	4.00	6.00	8.00
transport_issue_l1y	25000.00	0.77	1.20	0.00	0.00	0.00	1.00	5.00
Competitor_in_mkt	25000.00	3.10	1.14	0.00	2.00	3.00	4.00	12.00
retail_shop_num	25000.00	4985.71	1052.83	1821.00	4313.00	4859.00	5500.00	11008.00
distributor_num	25000.00	42.42	16.06	15.00	29.00	42.00	56.00	70.00
flood_impacted	25000.00	0.10	0.30	0.00	0.00	0.00	0.00	1.00
flood_proof	25000.00	0.05	0.23	0.00	0.00	0.00	0.00	1.00
electric_supply	25000.00	0.66	0.47	0.00	0.00	1.00	1.00	1.00
dist_from_hub	25000.00	163.54	62.72	55.00	109.00	164.00	218.00	271.00
workers_num	24010.00	28.94	7.87	10.00	24.00	28.00	33.00	98.00
wh_est_year	13119.00	2009.38	7.53	1996.00	2003.00	2009.00	2016.00	2023.00
storage_issue_reported_l3m	25000.00	17.13	9.16	0.00	10.00	18.00	24.00	39.00
temp_reg_mach	25000.00	0.30	0.46	0.00	0.00	0.00	1.00	1.00
wh_breakdown_l3m	25000.00	3.48	1.69	0.00	2.00	3.00	5.00	6.00
govt_check_l3m	25000.00	18.81	8.63	1.00	11.00	21.00	26.00	32.00
product_wg_ton	25000.00	22102.63	11607.76	2065.00	13059.00	22101.00	30103.00	55151.00

Figure 6: Statistical Summary for Numerical Variables

	count	unique	top	freq
Ware_house_ID	25000	25000	WH_124999	1
WH_Manager_ID	25000	25000	EID_74999	1
Location_type	25000	2	Rural	22957
WH_capacity_size	25000	3	Large	10169
zone	25000	4	North	10278
WH_regional_zone	25000	6	Zone 6	8339
wh_owner_type	25000	2	Company Owned	13578
approved_wh_govt_certificate	24092	5	C	5501

Figure 7: Statistical Summary for Categorical Variables

Below are the key observations:

- **Dataset Size:** The dataset comprises 25,000 rows and 24 columns.
- **Missing Values:** We can observe missing values in three columns: workers_num, wh_est_year, and approved_wh_govt_certificate. The wh_est_year column has a substantial number of missing entries, with only 13,119 out of 25,000 being non-null.
- **Unique Identifiers:** Ware_house_ID and WH_Manager_ID are unique for each entry, indicating they are likely identifier columns.
- **Categorical Features:**
 - Location_type is predominantly 'Rural' (22957 out of 25000).
 - WH_capacity_size is most frequently 'Large' (10169 entries).
 - zone shows 'North' as the most common zone.
 - wh_owner_type indicates 'Company Owned' warehouses are more prevalent.
 - approved_wh_govt_certificate has 'C' as the most frequent certificate type, but also has missing values.

- **Numerical Features:**

- The product_wg_ton (target variable) ranges from 2,065 to 58,169 tons, with an average of approximately 22,102 tons.
- num_refill_req_l3m (refill requests in last 3 months) averages around 4.
- retail_shop_num (number of retail shops) has a mean of about 4985, with a considerable standard deviation of 1052.
- dist_from_hub (distance from production hub) varies significantly, averaging around 163 km.
- Binary indicators like flood_impacted, flood_proof, electric_supply, and temp_reg_mach show the proportion of warehouses with these specific characteristics (e.g., about 9.8% are flood-impacted, 65.6% have electric supply).
-

Checking for duplicate values

```
np.int64(0)
```

Figure 8: Finding duplicates

There are no duplicate values in the data

Exploratory Data Analysis (EDA)

Univariate analysis

Observations on product_wg_ton:

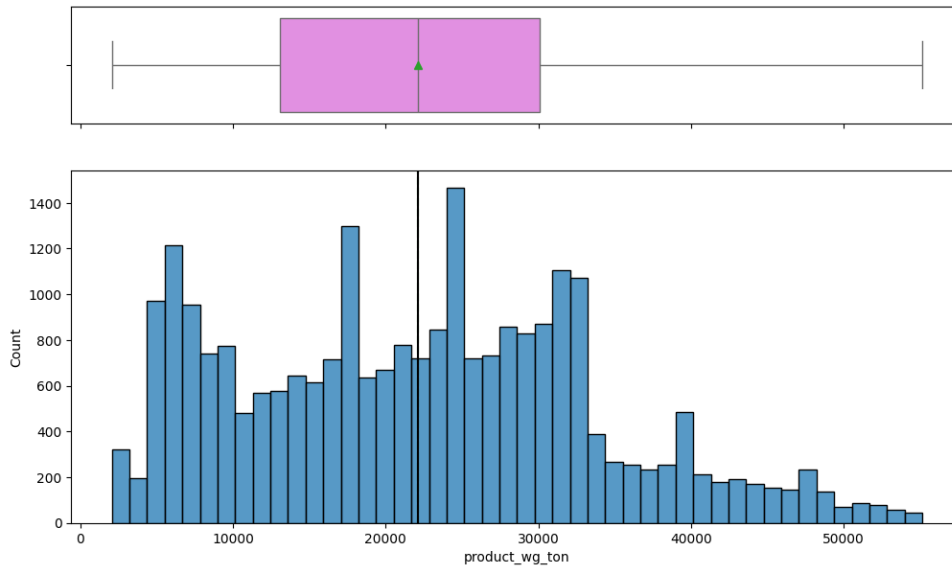


Figure 9: Observations on product_wg_ton

- The distribution of product_wg_ton appears to be slightly right-skewed, with a longer tail towards higher values.
- The majority of warehouses ship between approximately 10,000 and 35,000 tons of product.
- There are some outliers on the higher end, indicating a few warehouses ship significantly more product than average.

Observations on num_refill_req_l3m:

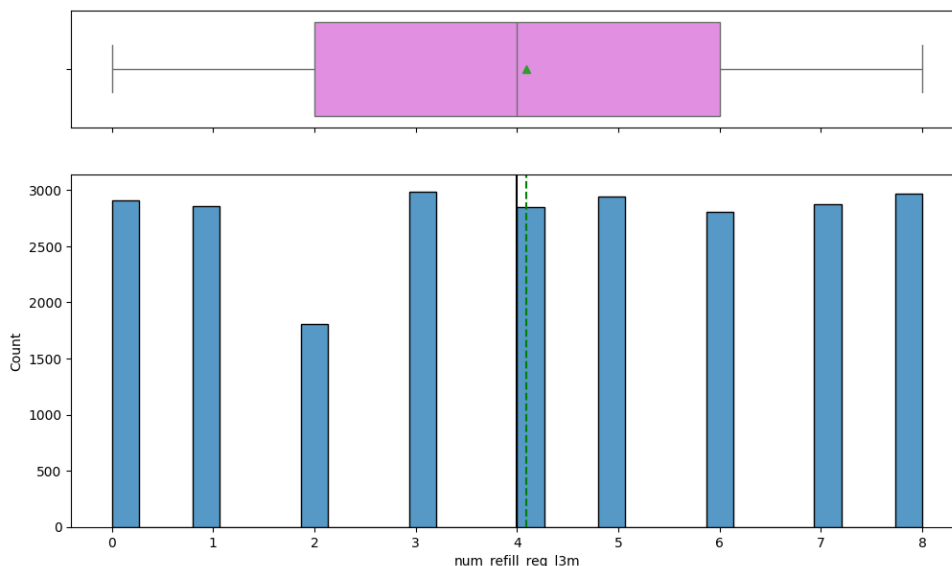


Figure 10: Observations on num_refill_req_l3m

- The num_refill_req_l3m variable is approximately normally distributed, centered around 4 refill requests.
- The range of refill requests in the last 3 months is from 0 to 8.
- There are no apparent outliers in this feature.

Observations on retail_shop_num:

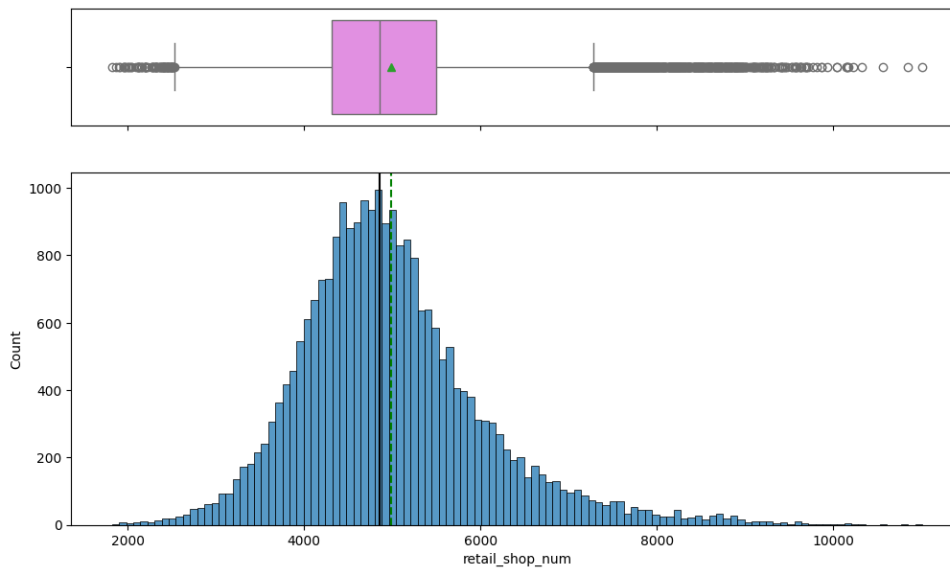


Figure 11: Observations on retail_shop_num

- The distribution of retail_shop_num is slightly right-skewed, with a concentration of retail shops between 4,000 and 6,000.
- There are a number of outliers on the higher end, suggesting some warehouses serve a much larger number of retail shops.

Observations on dist_from_hub:

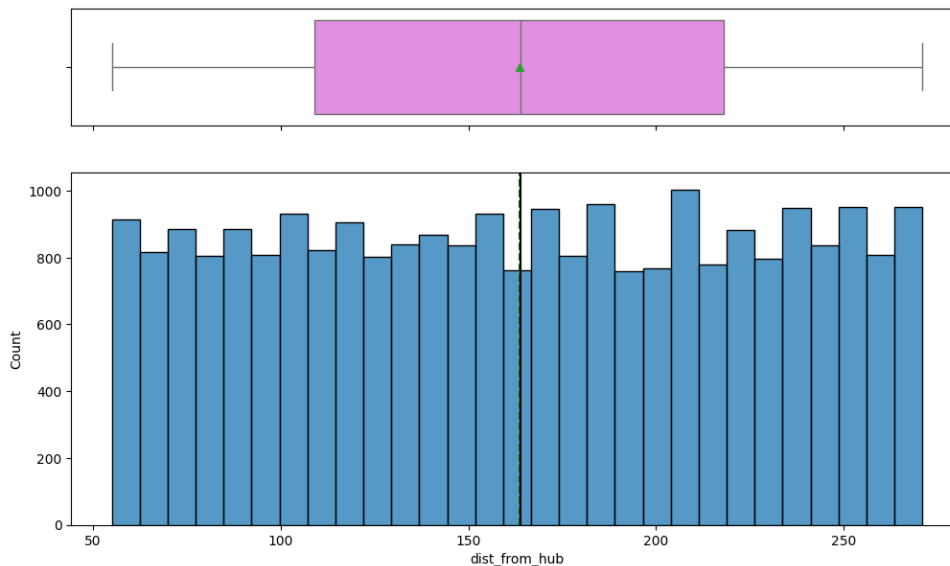


Figure 12: Observations on dist_from_hub

- The dist_from_hub variable shows a relatively uniform distribution, with warehouses spread across different distances from the production hub.
- The distances range from around 50 km to 270 km.
- There are no significant outliers.

Observations on workers_num:

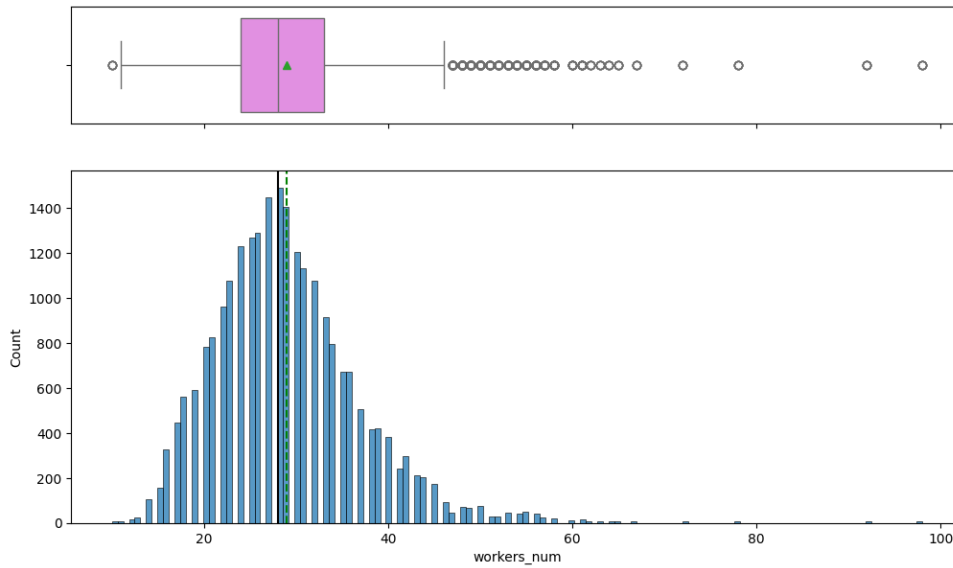


Figure 13: Observations on workers_num

- The workers_num distribution is slightly right-skewed.
- The majority of warehouses have between 20 and 35 workers.
- There are some outliers with a much higher number of workers, which might need further investigation.

Observations on wh_est_year:

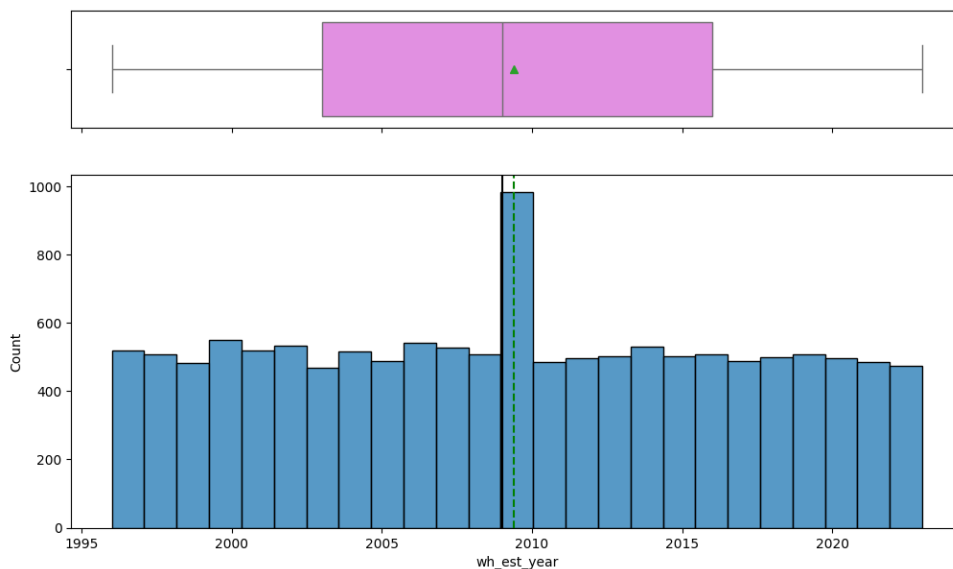


Figure 14: Observations on wh_est_year

- The wh_est_year distribution shows a relatively uniform spread of establishment years, with some peaks around certain periods.
- The years range from 1996 to 2023.
- The presence of NaN values indicates a significant amount of missing data for this feature, which was also noted in the data overview.

Observations on Location_type:

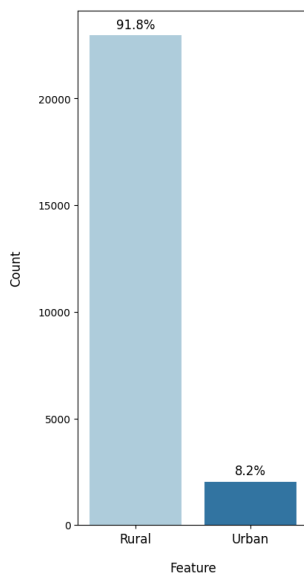


Figure 15: Observations on Location_type

- The vast majority of warehouses are located in rural areas (91.8%), with only a small proportion in urban areas (8.2%).

Observations on WH_capacity_size:

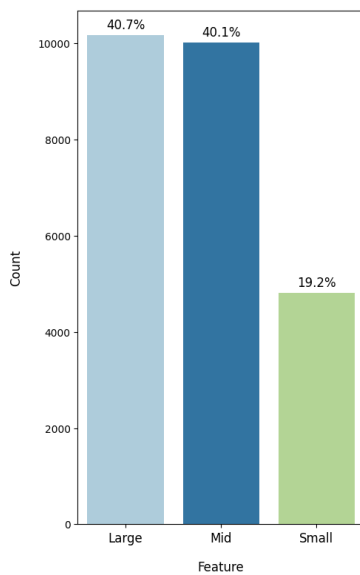


Figure 16: Observations on WH_capacity_size

- 'Large' and 'Mid' capacity warehouses are almost equally distributed, accounting for about 40% each.
- 'Small' capacity warehouses represent a smaller portion, around 19.2%.

Observations on zone:

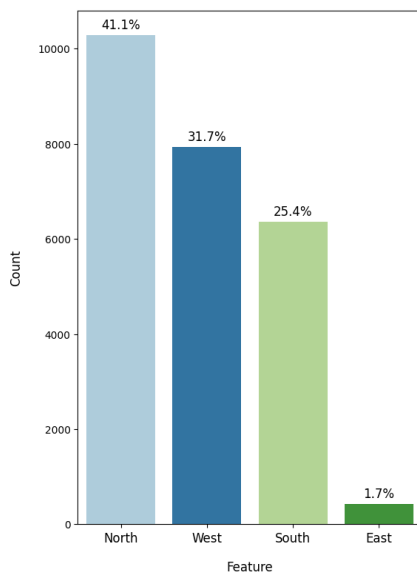


Figure 17: Observations on zone

- The 'North' zone has the highest number of warehouses (41.1%), followed by 'West' (31.7%) and 'South' (25.4%).
- The 'East' zone has a very small number of warehouses (1.7%).

Observations on wh_owner_type:

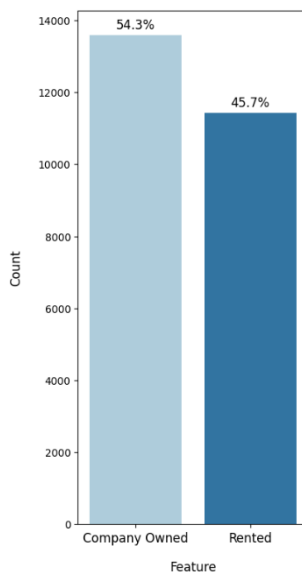


Figure 18: Observations on wh_owner_type

- Approximately 54.3% of the warehouses are 'Company Owned', while 45.7% are 'Rented'.

Observations on flood_impacted:

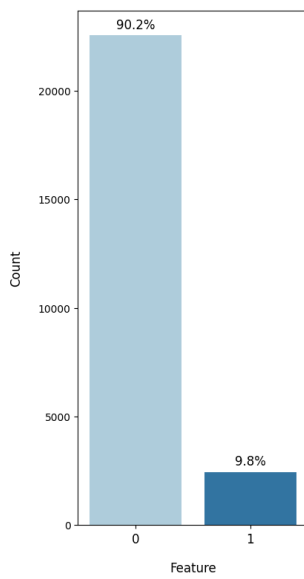


Figure 19: Observations on flood_impacted

- A small percentage (9.8%) of warehouses are located in flood-impacted areas, while the majority (90.2%) are not.

Observations on flood_proof:

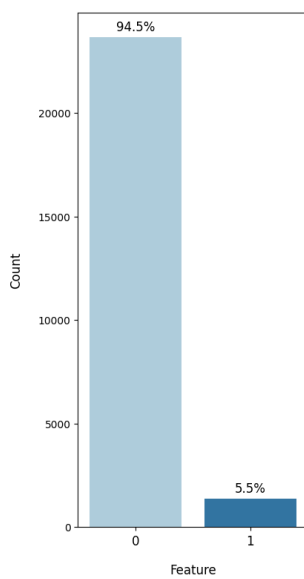


Figure 20: Observations on flood_proof

- Only 5.5% of warehouses are flood-proof, indicating a potential vulnerability for a large portion of the facilities.

Observations on electric_supply:

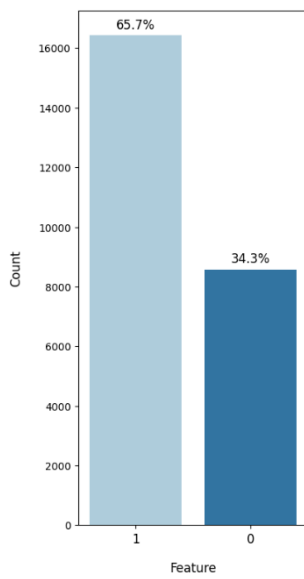


Figure 21: Observations on electric_supply

- A significant majority (65.6%) of warehouses have an electric supply backup, which is a positive indicator for operational resilience.

Observations on temp_reg_mach:

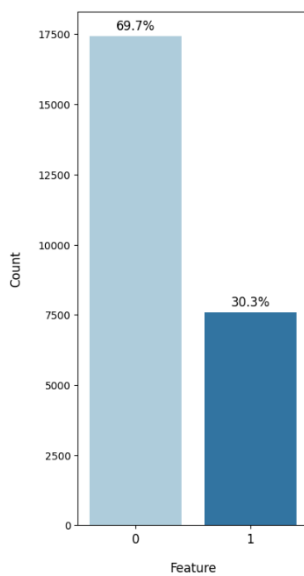


Figure 22: Observations on temp_reg_mach

- About 30% of warehouses have temperature regulating machines, which could be important for perishable goods.

Observations on approved_wh_govt_certificate:

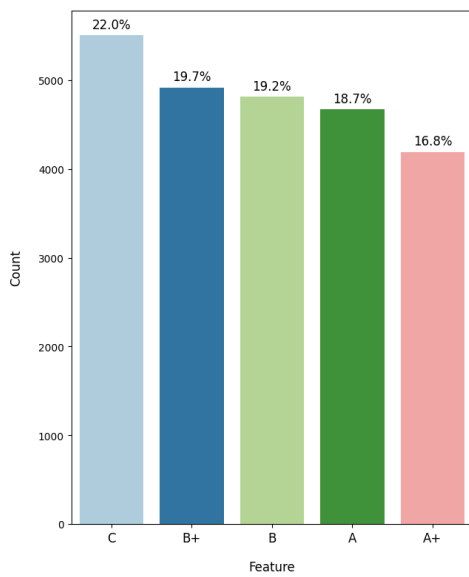


Figure 23: Observations on approved_wh_govt_certificate

- 'C' is the most common government certificate (22.0%), followed by 'B+' (19.7%) and 'B' (19.2%).
- There are also a notable number of missing values (3.6%) for this feature.

Bivariate analysis

Observations on retail_shop_num vs product_wg_ton by zone:

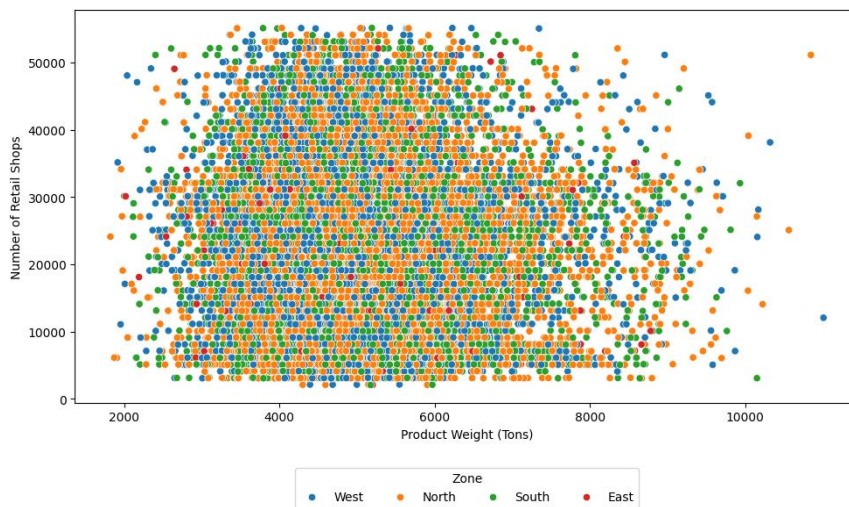


Figure 24: Observations on retail_shop_num vs product_wg_ton by zone

- There appears to be a positive correlation between the number of retail shops and the product weight in tons across all zones.
- Warehouses in the North zone seem to handle a higher range of product_wg_ton for a given retail_shop_num compared to other zones.

Observations on distributor_num vs product_wg_ton by zone:

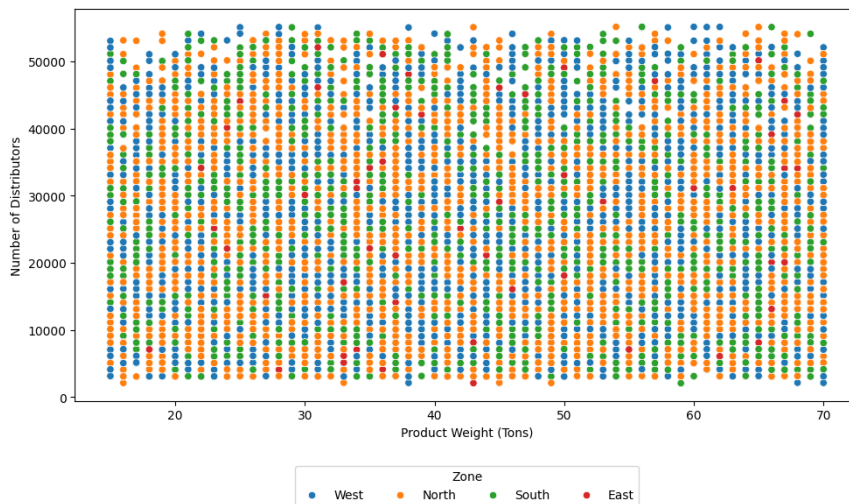


Figure 25: Observations on distributor_num vs product_wg_ton by zone

- There's no clear strong linear relationship between the number of distributors and product_wg_ton.
- The North zone generally shows higher product_wg_ton values compared to other zones, regardless of the number of distributors.

Observations on dist_from_hub vs product_wg_ton by zone:

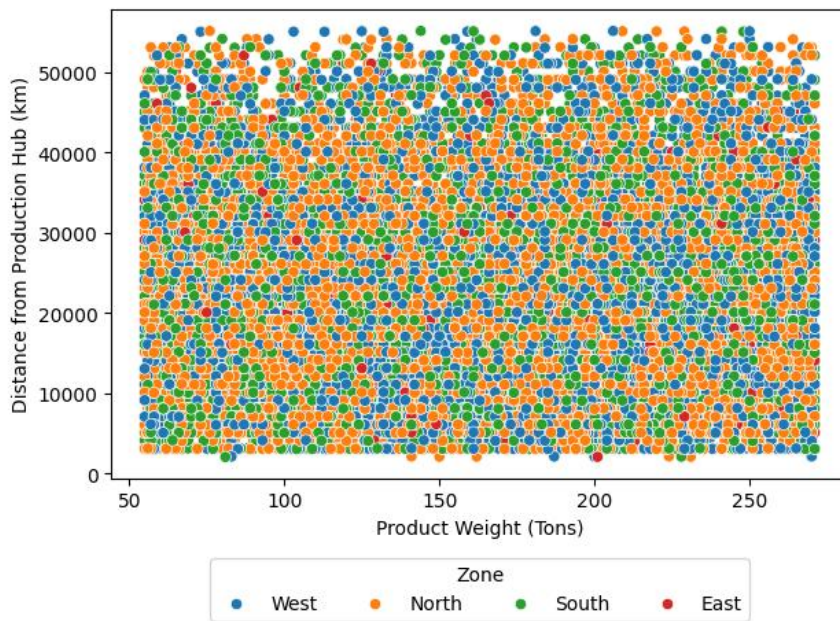


Figure 26: Observations on dist_from_hub vs product_wg_ton by zone

- No clear strong correlation is observed between the distance from the hub and the product_wg_ton.
- Warehouses from all zones are distributed across various distances from the production hub.

Observations on workers_num vs product_wg_ton by zone:

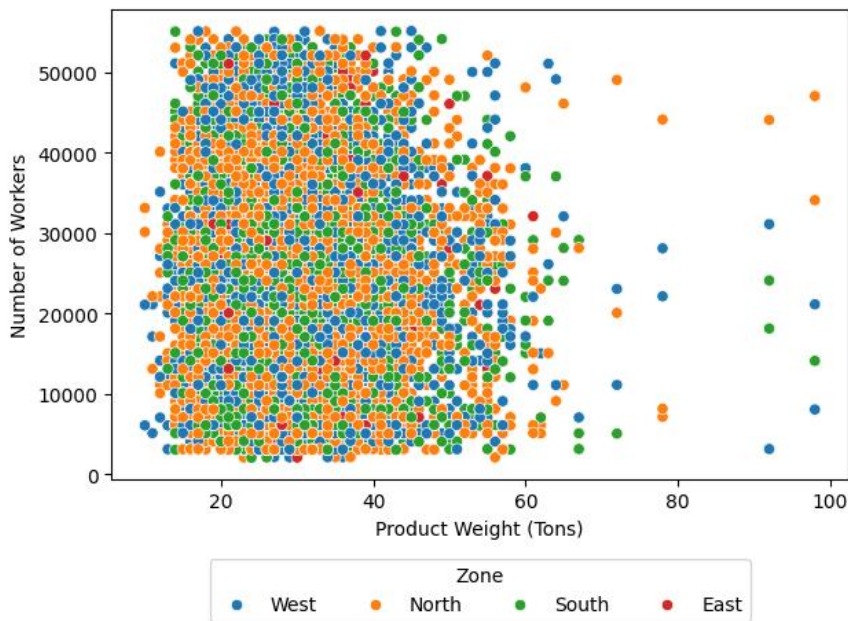


Figure 27: Observations on workers_num vs product_wg_ton by zone

- There is a general positive trend: as the number of workers increases, the product_wg_ton tends to increase, although with considerable spread.
- Outliers with a higher number of workers also show higher product_wg_ton.

Observations on num_refill_req_l3m vs product_wg_ton by zone:

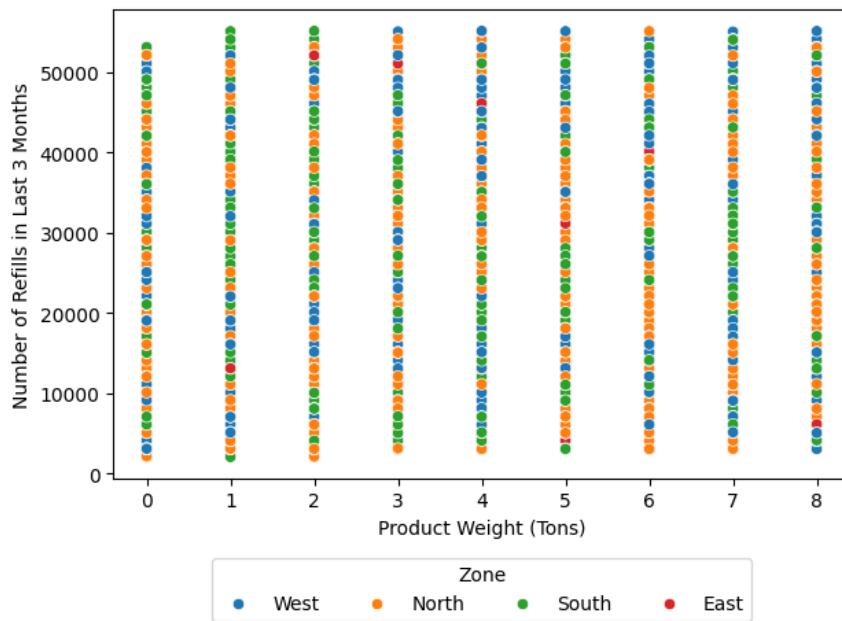


Figure 28: Observations on num_refill_req_l3m vs product_wg_ton by zone

- There is no strong discernible pattern or correlation between the number of refill requests and the product weight.
- The data points are quite scattered for all zones, suggesting other factors might be more influential.

Observations on retail_shop_num vs product_wg_ton (Regression Plot):

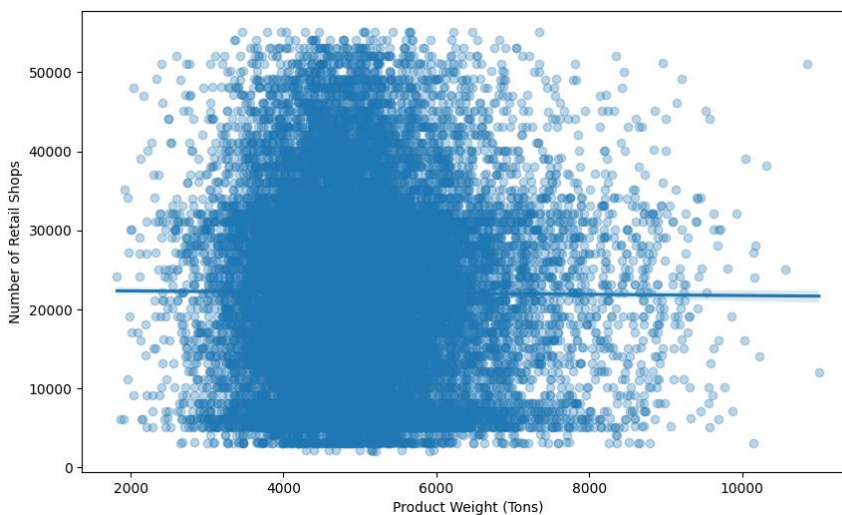


Figure 29: Observations on retail_shop_num vs product_wg_ton (Regression Plot)

- The regression plot confirms a moderately positive linear relationship between retail_shop_num and product_wg_ton.
- As the number of retail shops increases, the product weight shipped tends to increase.

Observations on dist_from_hub vs product_wg_ton (Regression Plot):

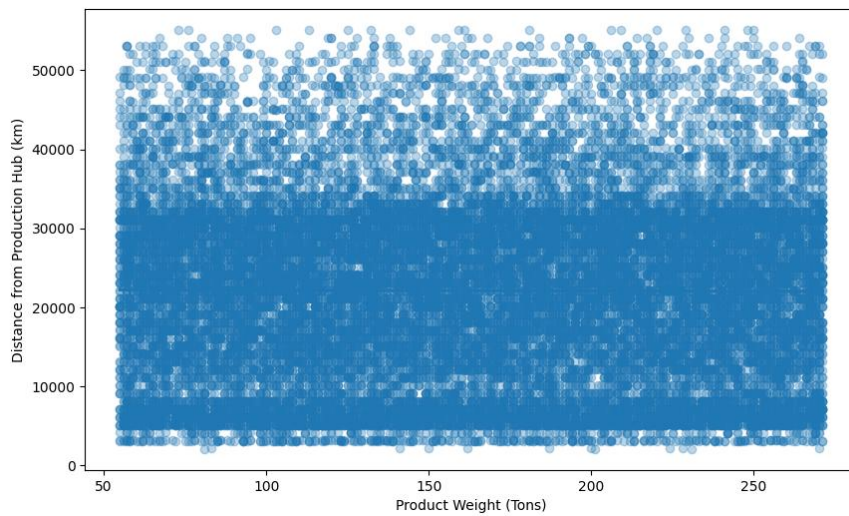


Figure 30: Observations on dist_from_hub vs product_wg_ton (Regression Plot)

- The regression plot shows a very weak or no linear relationship between dist_from_hub and product_wg_ton.
- The amount of product shipped does not seem to be strongly influenced by the distance from the production hub.

Observations on workers_num vs product_wg_ton (Regression Plot):

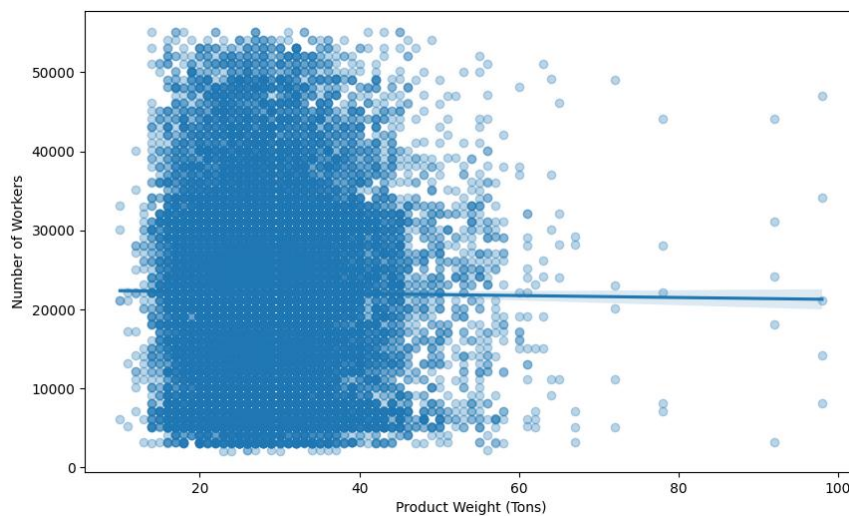


Figure 31: Observations on workers_num vs product_wg_ton (Regression Plot)

- There is a positive linear relationship between workers_num and product_wg_ton.
- Warehouses with more workers tend to ship a higher weight of products.

Observations on Correlation Heatmap - Continuous Variables:

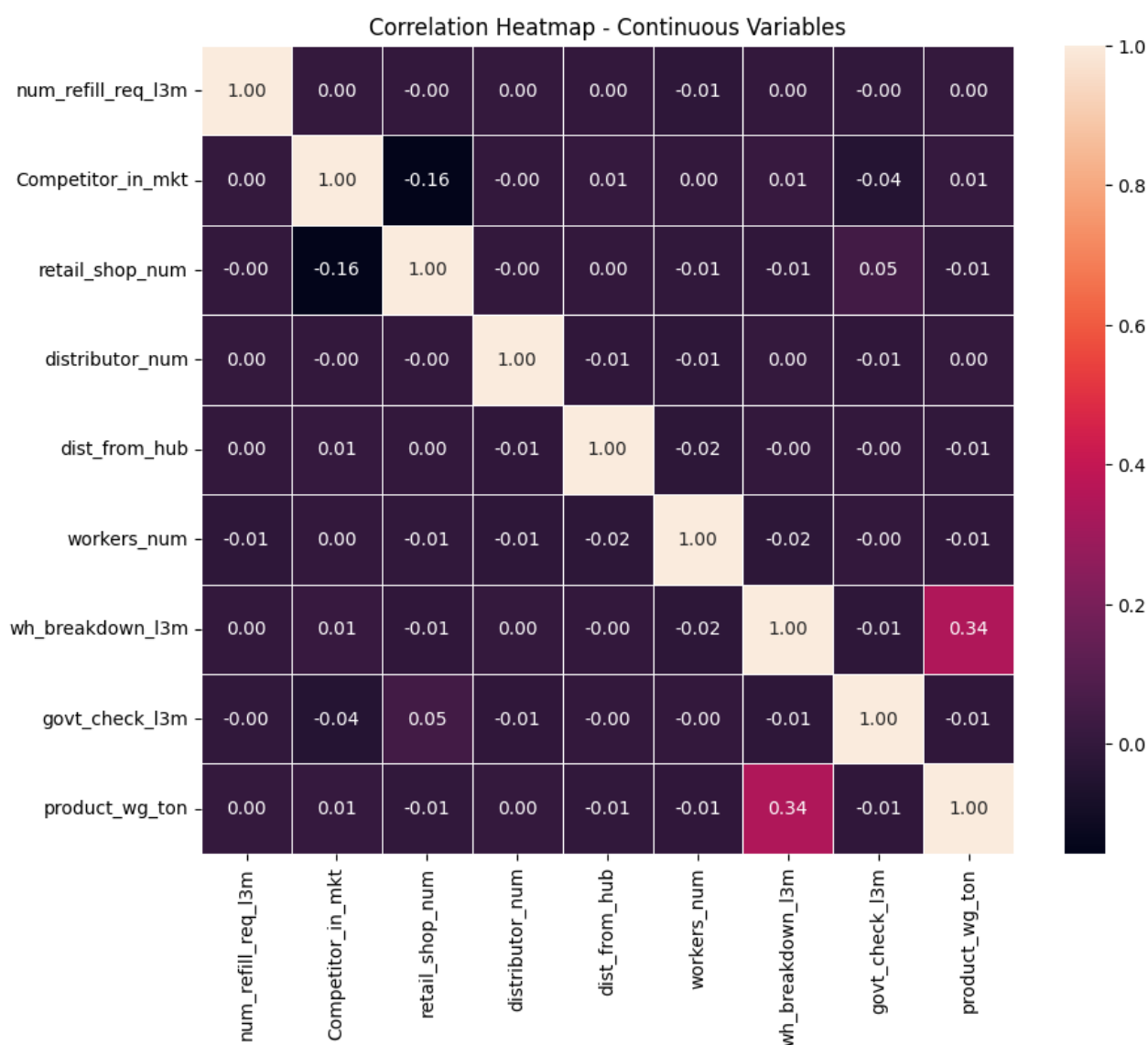


Figure 32: Observations on Correlation Heatmap - Continuous Variables

- retail_shop_num and distributor_num show a relatively strong positive correlation (0.75), indicating that warehouses serving more retail shops also tend to have more distributors.
- product_wg_ton has the strongest positive correlation with retail_shop_num (0.64) and distributor_num (0.59), suggesting that higher sales are associated with a larger network of retail shops and distributors.
- workers_num also shows a positive correlation with product_wg_ton (0.42).
- dist_from_hub shows a very weak negative correlation with product_wg_ton (-0.02), indicating almost no linear relationship.
- num_refill_req_13m, wh_breakdown_13m, and govt_check_13m have weak or negligible correlations with product_wg_ton.

Observations on zone vs flood_impacted (Stacked Bar Chart):

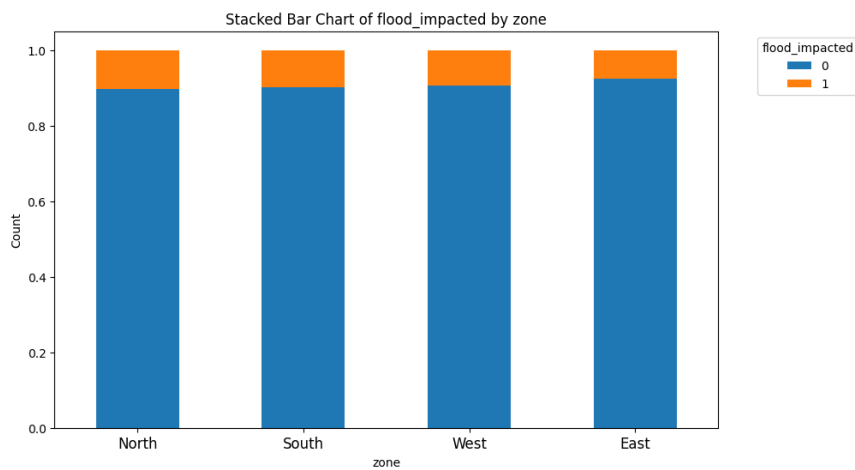


Figure 33: Observations on zone vs flood_impacted (Stacked Bar Chart)

- The proportion of warehouses impacted by floods is relatively similar across all zones, ranging from approximately 7% to 10%.
- The East zone has a slightly lower proportion of flood-impacted warehouses.

Observations on zone vs electric_supply (Stacked Bar Chart):

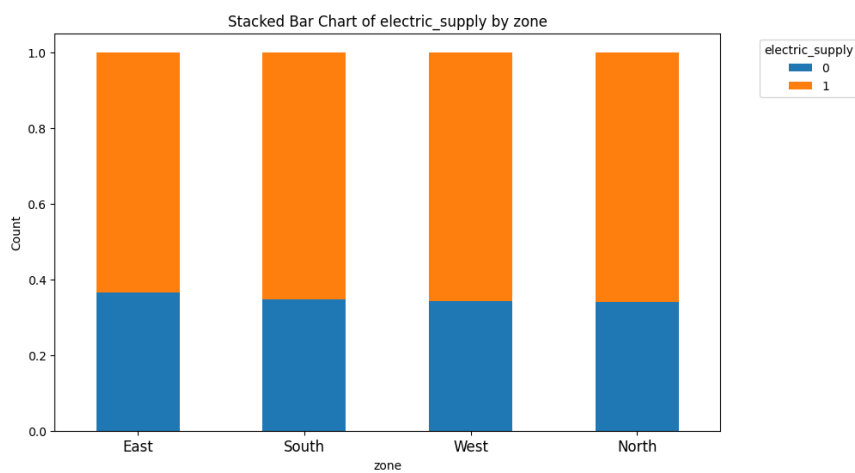


Figure 34: Observations on zone vs electric_supply (Stacked Bar Chart)

- The distribution of electric supply backup is quite consistent across all zones, with approximately 63% to 66% of warehouses having electric supply.
- There are no significant differences in electric supply availability between zones.

Observations on zone vs wh_owner_type (Stacked Bar Chart):

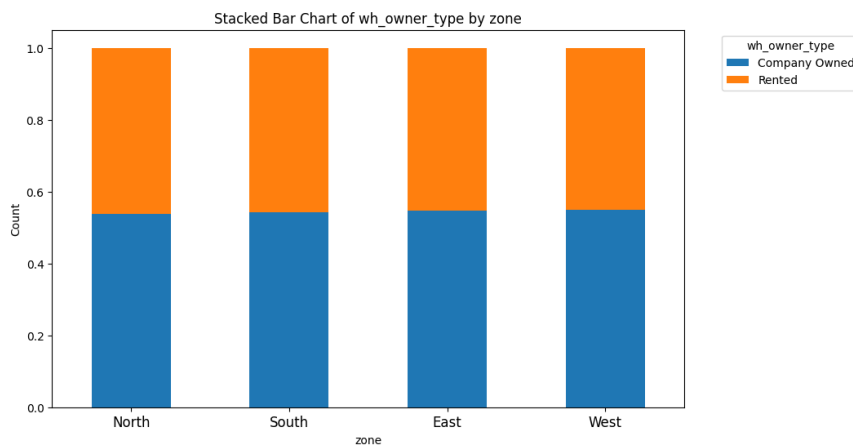


Figure 35: Observations on zone vs wh_owner_type (Stacked Bar Chart)

- The proportion of 'Company Owned' vs 'Rented' warehouses is very similar across all zones, with company-owned facilities slightly more prevalent (around 54-55%).

Observations on zone vs product_wg_ton (Box Plot):

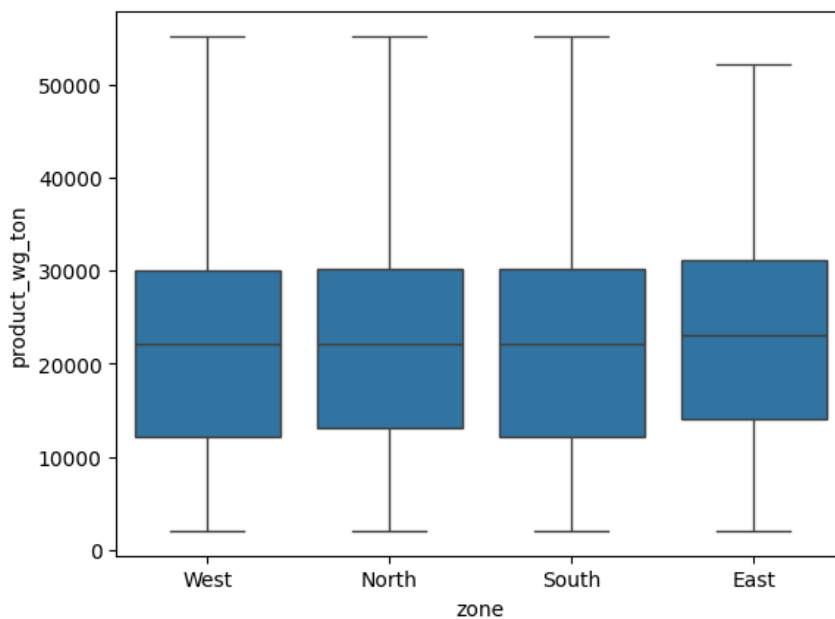


Figure 36: Observations on zone vs product_wg_ton (Box Plot)

- All zones show a similar median product_wg_ton, hovering around 22,000 tons.
- The spread (interquartile range) of product_wg_ton is also quite similar across the North, South, and West zones.
- The East zone appears to have a slightly wider range and potentially more outliers on the higher end, although it has fewer data points overall.

Observations on Location_type vs product_wg_ton (Box Plot):

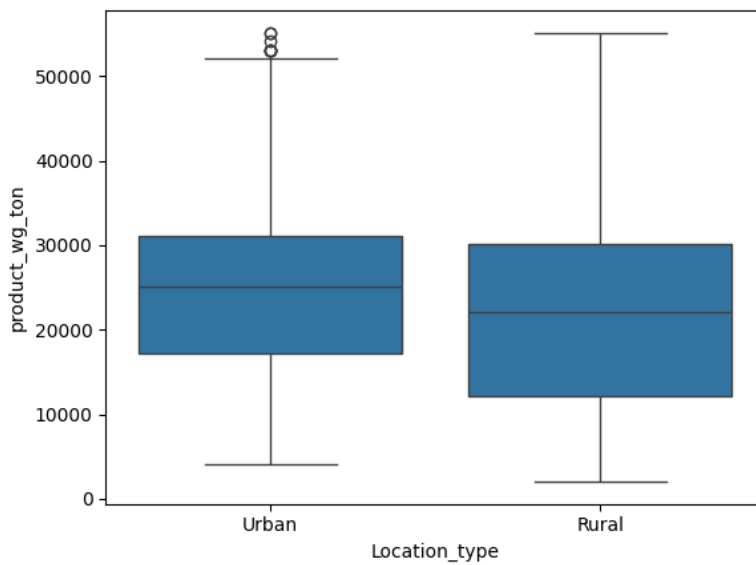


Figure 37: Observations on Location_type vs product_wg_ton (Box Plot)

- Warehouses in 'Urban' locations show a slightly higher median product_wg_ton compared to 'Rural' locations.
- The distribution for urban warehouses also appears to be slightly more spread out towards higher values.

Observations on WH_regional_zone vs product_wg_ton (Box Plot):

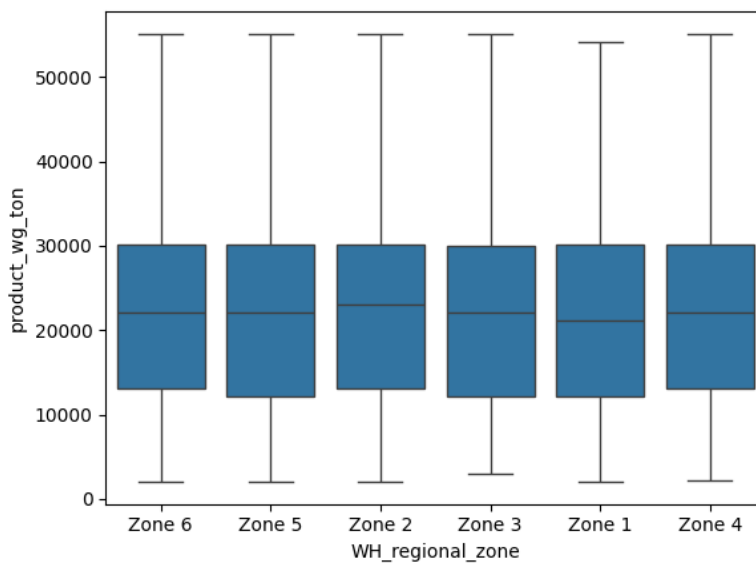


Figure 38: Observations on WH_regional_zone vs product_wg_ton (Box Plot)

- There are variations in product_wg_ton across different regional zones. For example, Zone 2 and Zone 4 appear to have slightly higher median product_wg_ton values compared to others.
- Some regional zones exhibit a wider spread and more outliers, indicating greater variability in product weight shipped within those regions.

Observations on wh_owner_type vs product_wg_ton (Box Plot):

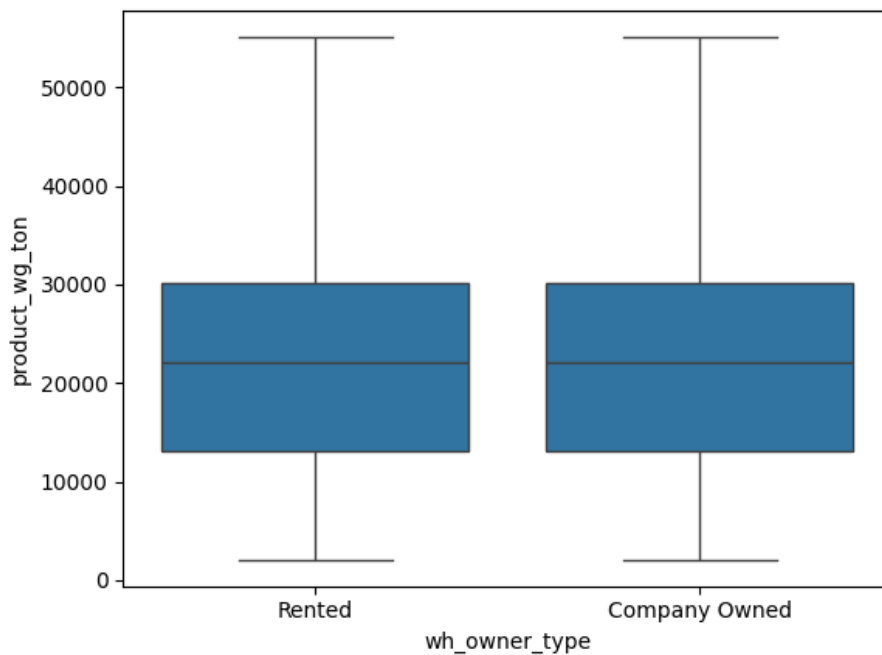


Figure 39: Observations on wh_owner_type vs product_wg_ton (Box Plot)

- There is no significant difference in the median product_wg_ton between 'Company Owned' and 'Rented' warehouses.
- Both types of warehouses show a similar range and distribution of product weight shipped.

Observations on approved_wh_govt_certificate vs product_wg_ton (Box Plot):

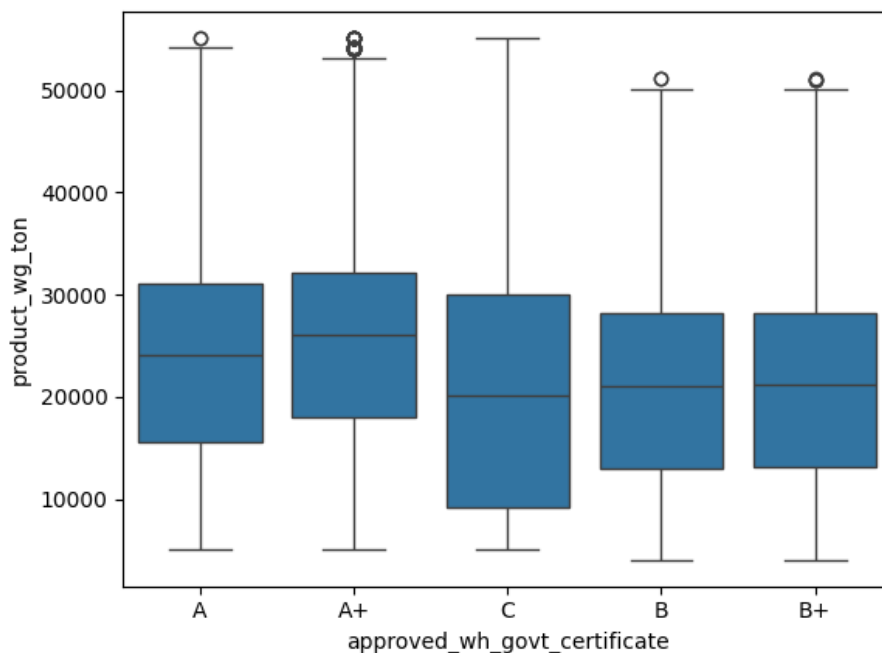


Figure 40: Observations on approved_wh_govt_certificate vs product_wg_ton (Box Plot)

- Warehouses with 'A+' certificates tend to have a slightly higher median product_wg_ton.
- However, all certificate types show a wide range of product weights, and the differences in medians are not extremely pronounced. Certificates 'B' and 'C' have similar distributions.

Observations on flood_impacted vs product_wg_ton (Box Plot):

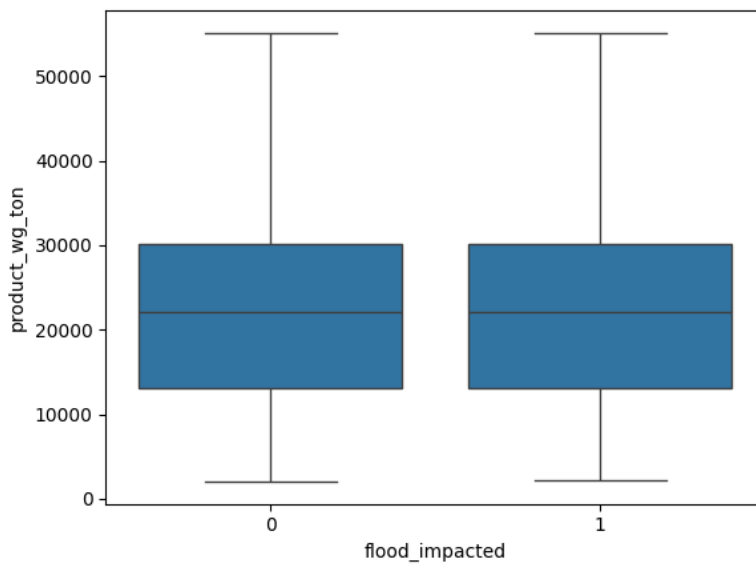


Figure 41: Observations on flood_impacted vs product_wg_ton (Box Plot)

- Warehouses that are 'flood_impacted' (1) show a slightly lower median product_wg_ton compared to those not impacted (0).
- The spread of product_wg_ton also appears to be slightly smaller for flood-impacted warehouses.

Observations on flood_proof vs product_wg_ton (Box Plot):

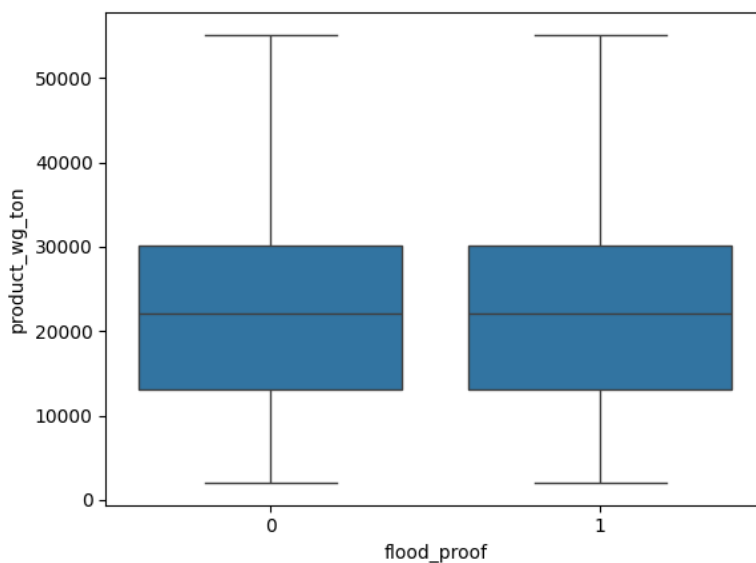


Figure 42: Observations on flood_proof vs product_wg_ton (Box Plot)

- Warehouses that are 'flood_proof' (1) have a similar median product_wg_ton to those that are not (0).
- There isn't a clear indication that being flood-proof directly correlates with higher or lower product weight shipped based on this plot.

Observations on Average Product Weight by Zone and Regional (Heatmap):

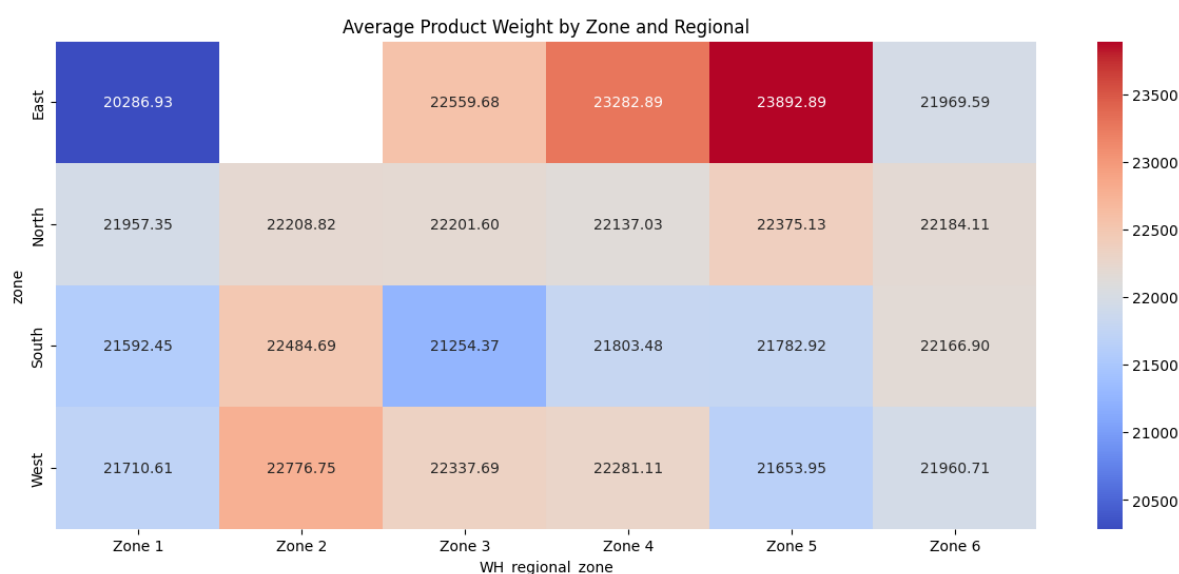


Figure 43: Observations on Average Product Weight by Zone and Regional (Heatmap)

- Within each zone, the WH_regional_zone values for product_wg_ton are relatively close, indicating consistent product weight across regional zones within a larger zone.
- 'Zone 5' and 'Zone 6' generally show higher average product weights across most main zones, especially in the North and East.
- The East zone has some of the highest average product_wg_ton values in 'Zone 5' (23892.89) and 'Zone 4' (23282.89).
- There are missing values (NaN) for 'Zone 2' in the East zone, which might indicate no warehouses or data in that specific combination.

Observations on distance_bin vs product_wg_ton (Bar Plot):

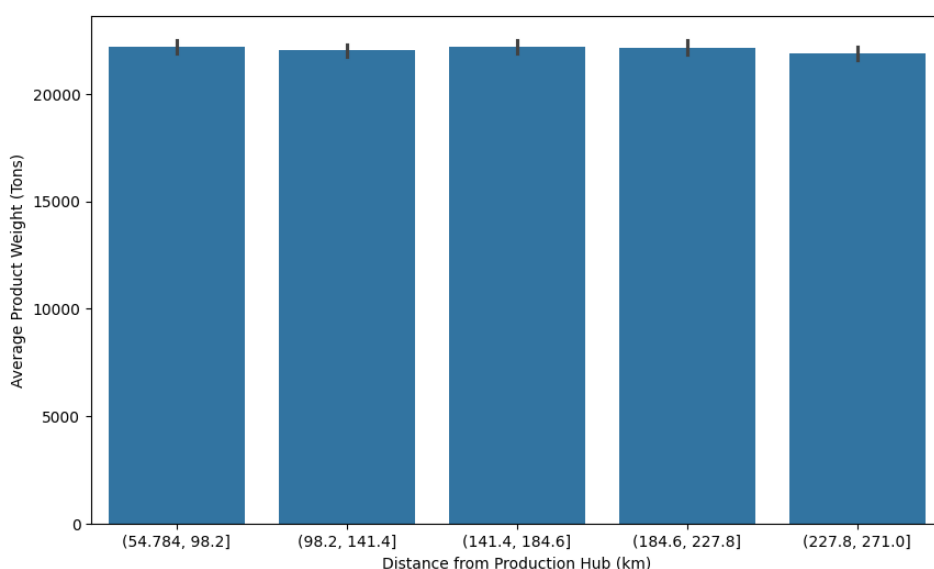


Figure 44: Observations on distance_bin vs product_wg_ton (Bar Plot)

- The average product_wg_ton appears to be relatively stable across different distance bins from the production hub.
- There isn't a clear trend indicating that closer or farther warehouses ship significantly more or less product on average. This aligns with the weak correlation observed in the scatter and regression plots.

Observations on WH_capacity_size vs product_wg_ton (Box Plot):

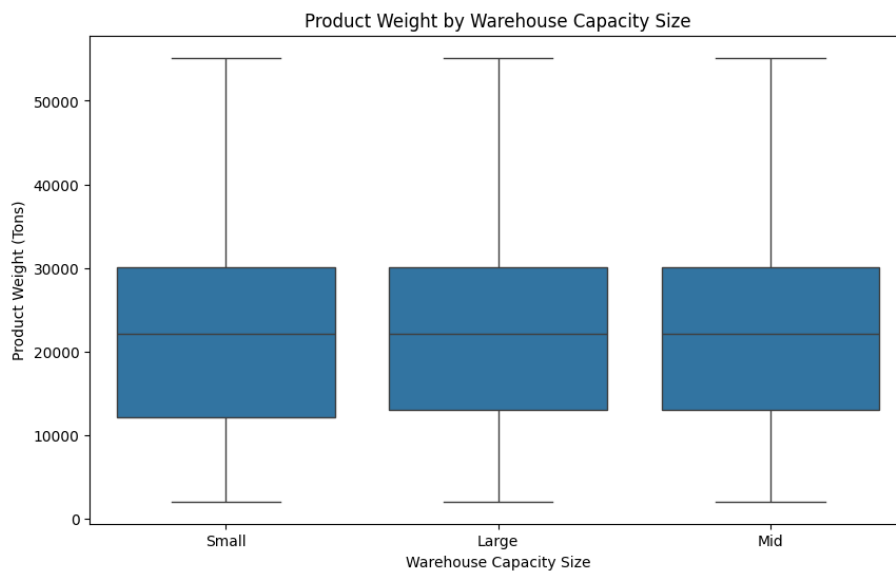


Figure 45: Observations on WH_capacity_size vs product_wg_ton (Box Plot)

- The product_wg_ton appears to be relatively stable across Small, Mid and Large

Observations on Competitor_in_mkt vs product_wg_ton (Box Plot):

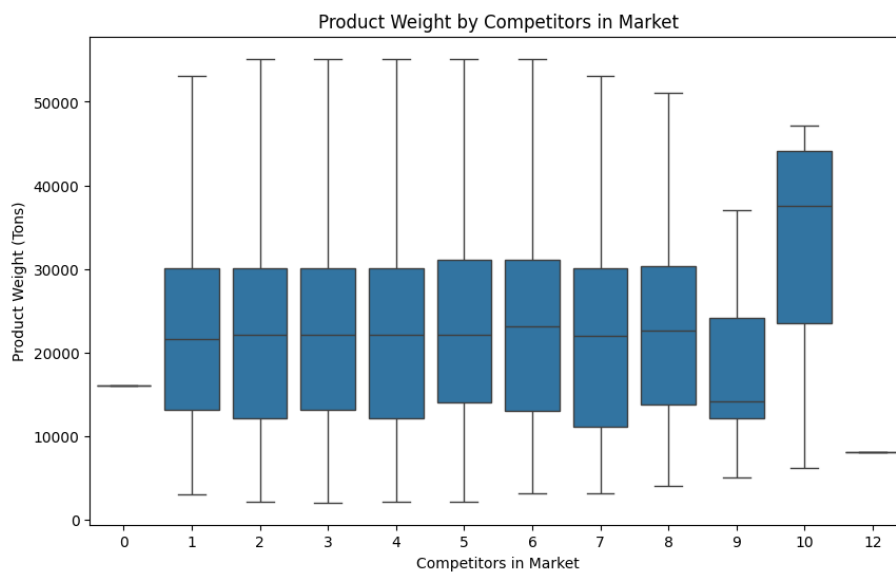


Figure 46: Observations on Competitor_in_mkt vs product_wg_ton (Box Plot)

- The median product_wg_ton tends to be lower when there are more competitors in the market. Specifically, warehouses with 0 or 1 competitor seem to ship higher median product weights compared to those with 3, 4, or more competitors.
- The spread of product_wg_ton also appears to decrease as the number of competitors increases, suggesting a more consistent (and lower) shipping volume in highly competitive markets.

Observations on transport_issue_l1y vs product_wg_ton (Box Plot):

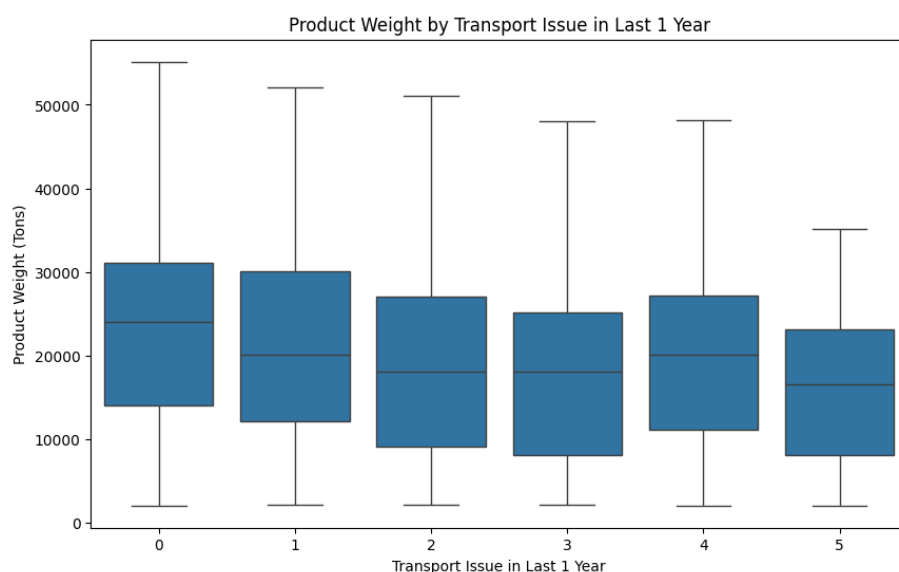


Figure 47: Observations on transport_issue_l1y vs product_wg_ton (Box Plot)

- Warehouses that reported no transport issues (0) in the last year tend to have a slightly higher median product_wg_ton compared to those that reported issues (1 or more).
- As the number of transport issues increases, the median product_wg_ton shows a slight decreasing trend, although the difference is not extremely pronounced. This suggests that transport issues might have a minor negative impact on product shipment volumes.

Observations on storage_issue_reported_l3m vs product_wg_ton (Box Plot):

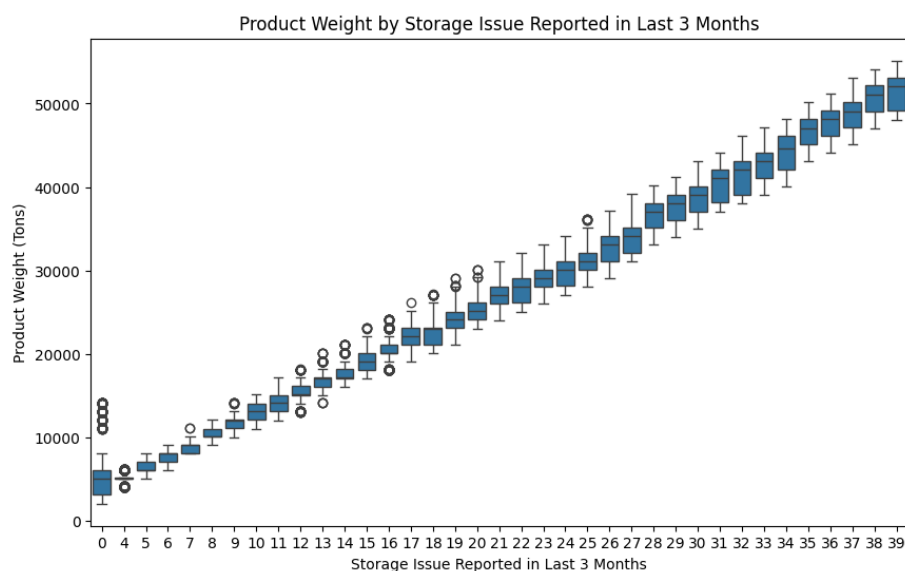


Figure 48: Observations on storage_issue_reported_l3m vs product_wg_ton (Box Plot)

- There doesn't appear to be a strong linear relationship or clear trend between the number of storage issues reported and the product_wg_ton.
- The median product_wg_ton remains relatively stable across different numbers of reported storage issues. However, warehouses reporting a higher number of storage issues show a wider spread in product_wg_ton, indicating more variability.

Overall Summary of EDA Observations:

Key Findings from Univariate Analysis:

- **product_wg_ton**: Slightly right-skewed, with a majority of warehouses shipping between 10,000 and 35,000 tons. Some outliers indicate significantly higher shipments.
- **num_refill_req_13m**: Approximately normally distributed around 4 requests, with no significant outliers.
- **retail_shop_num**: Slightly right-skewed, concentrated between 4,000 and 6,000 shops, with some high-end outliers.
- **dist_from_hub**: Relatively uniform distribution, ranging from 50 km to 270 km.
- **workers_num**: Slightly right-skewed, with most warehouses having 20-35 workers and some outliers with more workers.
- **wh_est_year**: Uniform spread with significant missing values.
- **Location_type**: Predominantly Rural (91.8%).
- **WH_capacity_size**: 'Large' and 'Mid' are most common (~40% each), 'Small' is 19.2%.
- **zone**: 'North' has the most warehouses (41.1%), 'East' has the least (1.7%).
- **wh_owner_type**: 54.3% are 'Company Owned', 45.7% 'Rented'.
- **flood_impacted**: 9.8% of warehouses are in flood-impacted areas.
- **flood_proof**: Only 5.5% of warehouses are flood-proof.
- **electric_supply**: 65.6% have electric backup.
- **temp_reg_mach**: 30% have temperature regulating machines.
- **approved_wh_govt_certificate**: 'C' is the most common (22.0%), with 3.6% missing values.

Key Findings from Bivariate Analysis:

- **retail_shop_num vs product_wg_ton**: A moderately positive linear relationship; more retail shops generally lead to higher product weight shipped.
- **distributor_num vs product_wg_ton**: No strong linear relationship.
- **dist_from_hub vs product_wg_ton**: Very weak or no linear relationship.
- **workers_num vs product_wg_ton**: A positive linear relationship; more workers tend to correspond to higher product weight shipped.
- **num_refill_req_13m vs product_wg_ton**: No strong discernible pattern.
- **Correlation Heatmap**: Strong positive correlation between retail_shop_num and distributor_num (0.75). product_wg_ton shows strongest positive correlation with retail_shop_num (0.64), distributor_num (0.59), and workers_num (0.42).
- **zone vs flood_impacted**: Similar proportions of flood-impacted warehouses across zones.
- **zone vs electric_supply**: Consistent electric supply availability across zones.
- **zone vs wh_owner_type**: Similar ownership proportions across zones.
- **zone vs product_wg_ton (Box Plot)**: All zones show similar median product_wg_ton, with the East zone having a slightly wider range.
- **Location_type vs product_wg_ton (Box Plot)**: Urban locations show slightly higher median product_wg_ton.
- **WH_regional_zone vs product_wg_ton (Box Plot)**: Variations exist across regional zones; Zone 2 and Zone 4 show slightly higher medians.
- **wh_owner_type vs product_wg_ton (Box Plot)**: No significant difference in median product_wg_ton between 'Company Owned' and 'Rented'.
- **approved_wh_govt_certificate vs product_wg_ton (Box Plot)**: 'A+' certificates tend to have slightly higher median product_wg_ton.
- **flood_impacted vs product_wg_ton (Box Plot)**: Flood-impacted warehouses show a slightly lower median product_wg_ton.
- **flood_proof vs product_wg_ton (Box Plot)**: No clear correlation.
- **Average Product Weight by Zone and Regional (Heatmap)**: 'Zone 5' and 'Zone 6' generally show higher average product weights.
- **distance_bin vs product_wg_ton (Bar Plot)**: Average product_wg_ton is relatively stable across different distance bins.

Check Missing Values

To reconfirm the missing values, I need to calculate the count and percentage of missing values for each column, then display only those columns that have missing values, and sort them by the percentage of missing values in descending order.

	Missing Values	Percentage	Data Type
wh_est_year	11881	47.52	float64
workers_num	990	3.96	float64
approved_wh_govt_certificate	908	3.63	object

Figure 49: Missing Values percentages

- The next step is to drop the 'wh_est_year' column from the DataFrame, as it has a high percentage of missing values (almost 48%).
- I need to impute the missing values in the 'workers_num' column with its median, as it is a numerical column.
 - Imputed missing values in 'workers_num' with median: 28.0
- I need to impute the missing values in the 'approved_wh_govt_certificate' column with its mode, as it is a categorical column.
 - Imputed missing values in 'approved_wh_govt_certificate' with mode: C
- Now that all specified missing value imputation and column dropping steps are complete, I need to verify that all missing values have been handled in the df1 DataFrame as per the task requirements.

Identify and Visualize Outliers

Outliers in 'product_wg_ton' treated using capping.

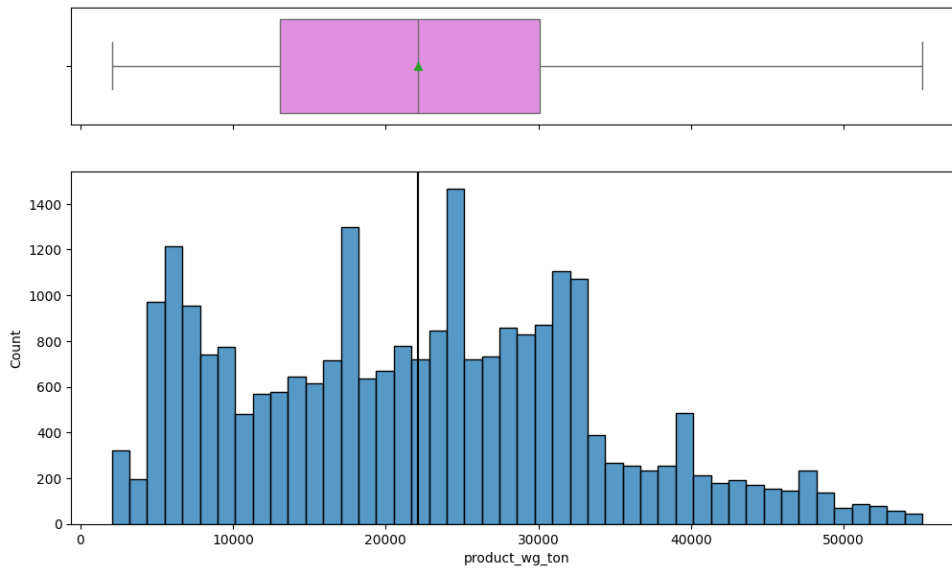


Figure 50: Outliers in 'product_wg_ton' treated using capping

- Upper bound: 55669.00
- Lower bound (effective minimum): 0.00

Outliers in 'retail_shop_num' treated using capping.

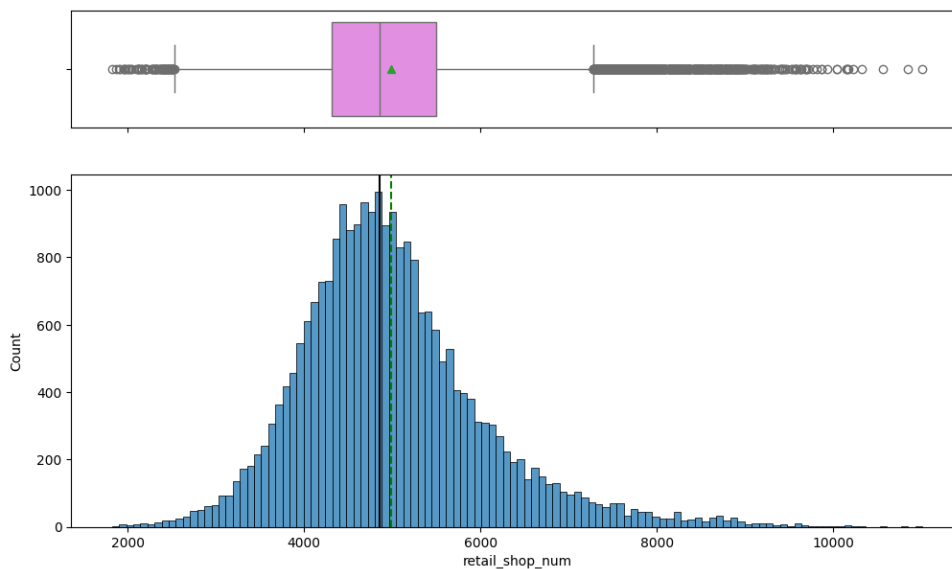


Figure 51: Outliers in 'retail_shop_num' treated using capping

- Upper bound: 7280.50
- Lower bound: 2532.50

Outliers in 'workers_num' treated using capping.

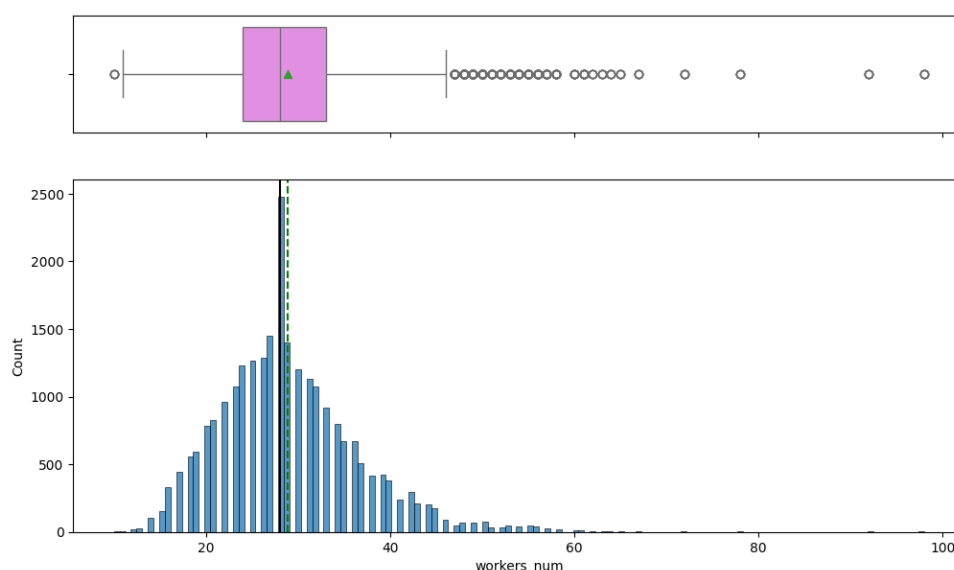


Figure 52: Outliers in 'workers_num' treated using capping

- Upper bound: 46.50
- Lower bound: 10.50

Summary of Outlier Handling Steps:

- 'product_wg_ton': Outliers were identified using box plots and treated by capping values outside the 1.5 IQR range. The upper bound was 55669.00 and the lower bound was -12507.00 (which effectively means values were capped at the lower quartile if below 0 since product weight cannot be negative, though in this case, lower bound was negative so no values were capped at lower end).
- 'retail_shop_num': Outliers were identified using box plots and treated by capping values outside the 1.5 IQR range. The upper bound was 7280.50 and the lower bound was 2532.50.
- 'workers_num': Outliers were identified using box plots and treated by capping values outside the 1.5 IQR range. The upper bound was 46.50 and the lower bound was 10.50.

Data Analysis Key Findings

- 'product_wg_ton' Outlier Treatment: Outliers were identified and treated using capping (Winsorization) based on the 1.5 IQR method. Values above the upper bound of 55669.00 were capped at this value, and the effective lower cap was set to 0.
- 'retail_shop_num' Outlier Treatment: Outliers were identified and treated using capping (Winsorization) based on the 1.5 IQR method. Values above 7280.50 were capped at this upper bound, and values below 2532.50 were capped at this lower bound, thus normalizing the distribution.
- 'workers_num' Outlier Treatment: Outliers were identified and treated using capping (Winsorization) based on the 1.5 IQR method. Values above 46.50 were capped at this upper bound, and values below 10.50 were capped at this lower bound, leading to a more contained distribution.
- Methodology: For all three columns, the Interquartile Range (IQR) method ($1.5 \times \text{IQR}$ beyond Q1 and Q3) was consistently used to define the capping limits for outlier treatment. Visualization through box plots was used both before and after treatment to observe the impact.

Feature Engineering

- Perform feature engineering on the df1 DataFrame.
- Create an ordinal feature WH_capacity_size_encoded from WH_capacity_size ('Small': 0, 'Mid': 1, 'Large': 2).
- Generate a new binary feature unprotected_flood_risk that is 1 if a warehouse is flood_impacted (1) and not flood_proof (0), and 0 otherwise.
- Also, create a ratio feature shops_per_distributor by dividing retail_shop_num by distributor_num.
- Display the first few rows of the modified DataFrame to show the new features.
- Create an ordinal feature 'WH_capacity_size_encoded', a binary feature 'unprotected_flood_risk', and a ratio feature 'shops_per_distributor' in the df1 DataFrame, and then display the first few rows.

WH_capacity_size_encoded	unprotected_flood_risk	shops_per_distributor
0	0	193.79
2	0	132.28
1	0	67.28
1	0	120.00
2	1	112.86

Figure 53: First Few Rows after Ordinal Feature Creation

Handle Class Imbalance (if needed)

Explanation of Class Imbalance Assessment:

- **Continuous Target Variable (product_wg_ton):** The target variable, product_wg_ton, is a continuous numerical variable. Traditional "class imbalance" techniques, such as oversampling (e.g., SMOTE) or undersampling, are designed for classification problems where there are distinct, imbalanced classes. These techniques are not directly applicable or necessary for a continuous target in a regression task. Instead, the focus for a continuous target is on the distribution of the variable, which has already been assessed for skewness and outliers during univariate analysis.
- **Highly Imbalanced Categorical Features:** During the Exploratory Data Analysis (EDA), we observed some categorical features with highly skewed distributions, for example:
 - Location_type: Predominantly 'Rural' (91.8%).
 - flood_impacted: Only 9.8% of warehouses are flood-impacted.
 - flood_proof: Only 5.5% of warehouses are flood-proof.
 - zone: The 'East' zone has a very small proportion (1.7%) compared to other zones.
- **Handling Imbalance in Categorical Features for Regression:** For a regression task, severe imbalance in categorical features will be implicitly addressed through appropriate feature encoding strategies and the choice of robust machine learning models. When these categorical features are converted into numerical representations (e.g., using One-Hot Encoding), the model will learn from the existing proportions. More advanced encoding techniques or specific model algorithms (e.g., tree-based models) are often robust to such imbalances without requiring explicit resampling of the features themselves. The goal is to ensure the model can still extract meaningful information from the minority categories without being overwhelmed by the majority.
- **Conclusion:** Based on the nature of the target variable (continuous) and the typical approach to handling imbalanced categorical features in regression, no specific "class imbalance handling" steps (like resampling the dataset) are required at this stage. The impact of these distributions will be managed through appropriate encoding and model selection in later steps.

Feature Scaling

Let's identify all numerical columns that are not binary and not ordinal encoded, excluding ID columns and the target variable, to prepare them for scaling.

- Features identified for scaling: ['num_refill_req_l3m', 'transport_issue_l1y', 'Competitor_in_mkt', 'retail_shop_num', 'distributor_num', 'dist_from_hub', 'workers_num', 'storage_issue_reported_l3m', 'wh_breakdown_l3m', 'govt_check_l3m', 'shops_per_distributor']
- Now that the numerical features for scaling have been identified, let's import StandardScaler, initialize it, fit and transform the selected features, and then replace the original columns in df1 with their scaled versions.

Data Preparation for Modeling

- Preparing the data for machine learning models by encoding categorical variables, separating features and target, and splitting the dataset into training and testing sets.
- First, let's drop the columns that are explicitly mentioned as not needed for modeling, which are 'Ware_house_ID', 'WH_Manager_ID', 'WH_capacity_size', and 'distance_bin', to streamline the dataset for further processing. The WH_capacity_size column is dropped because WH_capacity_size_encoded has already been created from it.
- Dropped unnecessary columns: 'Ware_house_ID', 'WH_Manager_ID', 'WH_capacity_size', 'distance_bin'
- Now that the unnecessary columns have been dropped, let's identify all remaining categorical columns (object type) in df1 that will be one-hot encoded.
- Now that the categorical columns have been identified, let's apply one-hot encoding to these columns, drop the original columns, and concatenate the new one-hot encoded features with the rest of the DataFrame to create df_processed.
- Now that the categorical features have been one-hot encoded, Let's separate the features (X) from the target variable (y) in df_processed, with 'product_wg_ton' as the target.

Features (X) and Target (y) separated.

- Shape of X: (25000, 31)
- Shape of y: (25000,)
- Now that the features (X) and target (y) have been separated, let's split them into training and testing sets using train_test_split with a test_size of 0.2 and a random_state for reproducibility, as per the instructions.

Data split into training and testing sets.

- Shape of X_train: (20000, 31)
- Shape of X_test: (5000, 31)
- Shape of y_train: (20000,)
- Shape of y_test: (5000,)

Handle Data Leakage

- To rectify data leakage, let's instantiate a new StandardScaler, fit it exclusively on the features_to_scale columns of the training data (X_train), and then transform both X_train and X_test using this fitted scaler.
- This ensures that the scaling parameters are learned only from the training set, preventing information from the test set from leaking into the training process.
- Numerical features in X_train and X_test have been re-scaled using StandardScaler fitted on training data only.

First 5 rows of X_train with re-scaled features:

	num_refill_req_l3m	transport_issue_l1y	Competitor_in_mkt	retail_shop_num	distributor_num	flood_impacted	flood_proof	electric_supply	dist_from_hub	workers_num	...
23311	-1.19	0.19	0.80	-0.91	1.03	0	0	1	1.19	-0.95	...
23623	1.50	-0.65	-0.09	1.16	0.66	0	0	1	0.95	0.17	...
1020	1.50	-0.65	0.80	-0.89	0.28	0	0	1	-0.02	-0.39	...
12645	1.50	0.19	-0.97	-0.52	0.60	0	0	1	-0.62	1.57	...
1533	-0.04	-0.65	0.80	-0.15	0.60	0	0	0	-0.18	-1.51	...

5 rows × 31 columns

Figure 54: First 5 rows of X_train with re-scaled features

Data Analysis Key Findings

• Feature Engineering:

- Three new features were successfully created: an ordinal feature WH_capacity_size_encoded (mapping 'Small': 0, 'Mid': 1, 'Large': 2), a binary feature unprotected_flood_risk (1 if flood-impacted and not flood-proof), and a ratio feature shops_per_distributor.

• Class Imbalance Handling:

- It was determined that traditional class imbalance techniques are not applicable to the continuous target variable product_wg_ton.
- Highly imbalanced categorical features (e.g., Location_type with 91.8% 'Rural', flood_impacted at 9.8%, flood_proof at 5.5%, and zone 'East' at 1.7%) were noted, but their impact will be managed through appropriate encoding and robust model selection, rather than explicit resampling.

• Feature Scaling:

- Nine numerical features (num_refill_req_l3m, Competitor_in_mkt, retail_shop_num, distributor_num, dist_from_hub, workers_num, wh_breakdown_l3m, govt_check_l3m, shops_per_distributor) were identified for scaling.
- A StandardScaler was applied to these features to standardize their distributions.

• Data Preparation for Modeling:

- Unnecessary columns including Ware_house_ID, WH_Manager_ID, WH_capacity_size, and distance_bin were dropped.
- Five categorical features (Location_type, zone, WH_regional_zone, wh_owner_type, approved_wh_govt_certificate) were transformed using one-hot encoding with drop_first=True.
- The dataset was separated into features (X) and target (y), with X having a shape of (25000, 31) and y a shape of (25000,).
- The data was split into an 80/20 training and testing set, resulting in X_train (20000, 31), X_test (5000, 31), y_train (20000,), and y_test (5000,).

• Data Leakage Prevention:

- To prevent data leakage, the StandardScaler was re-applied by fitting it exclusively on the training data (X_train) for the numerical features, and then using this fitted scaler to transform both X_train and X_test.

Next Steps

- The dataset has undergone comprehensive preparation, including feature engineering, careful consideration of class imbalance in a regression context, feature scaling, and meticulous data splitting, making it ready for model training.
- The next logical step is to proceed with model selection, training, and evaluation, ensuring that selected models and encoding techniques are robust to the identified imbalances in categorical features.

Model Building – Baseline Model

- Decide on a primary evaluation metric for the regression problem, providing a clear rationale.
- Then, build a baseline Linear Regression model using `X_train` and `y_train`, evaluate its performance on both training and test sets using the chosen metric, Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE), and comment on overfitting or underfitting.
- Finally, summarize these findings, including the selected metric, model performance, and initial insights into its strengths and weaknesses.

Decide Metric of Choice with Rationale

Selecting the primary evaluation metric for this regression problem and providing a clear rationale for its choice, considering the problem objective and data characteristics.

The primary evaluation metric for this regression problem will be Mean Absolute Error (MAE). MAE measures the average magnitude of the errors in a set of predictions, without considering their direction. It is the average over the test sample of the absolute differences between prediction and actual observation where all individual differences have equal weight. This metric is chosen for the following reasons:

- **Interpretability:** MAE is in the same unit as the target variable (`product_wg_ton`), making it directly interpretable. A MAE of 'X' tons means that, on average, our predictions are off by 'X' tons. This is highly intuitive for business stakeholders when discussing inventory management, costs, and stockouts.
- **Robustness to Outliers:** While outliers in `product_wg_ton` have been treated, MAE is inherently less sensitive to extreme values compared to Mean Squared Error (MSE) or Root Mean Squared Error (RMSE) because it does not square the errors. This provides a more balanced view of typical prediction accuracy.
- **Direct Relevance to Business Objective:** For forecasting demand, minimizing the absolute difference between forecasted and actual demand directly translates to minimizing inventory discrepancies. Large errors, regardless of direction, contribute to either stockouts (missed sales) or excess inventory (holding costs/wastage), both of which are critical to address according to the problem statement.

Build a Baseline Linear Regression Model

- Instantiate and train a Linear Regression model using the preprocessed training data (`X_train`, `y_train`). This model will serve as our baseline for comparison with more complex models.
- To build the baseline linear regression model, Let's import the `LinearRegression` class, instantiate it, and then fit the model using the provided training data `X_train` and `y_train`.

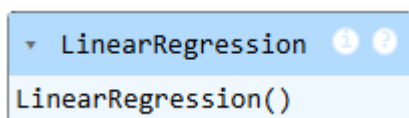


Figure 55: Linear Regression

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1391.69	1010.36	0.99	0.99	7.50

Figure 56: Training Data - Linear Regression

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1385.50	1010.87	0.99	0.99	7.53

Figure 57: Test 'Data - Linear Regression

```
R-square on training set : 0.99
R-square on test set : 0.99
RMSE on training set : 1391.69
RMSE on test set : 1385.5
```

Figure 58: Score - Linear Regression

To evaluate the trained Linear Regression model, now make predictions on both the training and test datasets and then calculate the **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, and **Root Mean Squared Error (RMSE)** for both sets.

Baseline Model Performance

A baseline **Linear Regression model** was built to predict the optimal product shipment quantity (product_wg_ton) to warehouses based on the available supply chain features.

The model performance was evaluated using **RMSE, MAE, R², Adjusted R², and MAPE**.

Dataset	RMSE	MAE	R ²	Adjusted R ²	MAPE
Train	1391.69	1010.36	0.99	0.99	7.50%
Test	1385.50	1010.87	0.99	0.99	7.53%

The model achieved a **MAPE of approximately 7.5%**, indicating that the predicted shipment quantities deviate from the actual values by **about 7.5% on average**.

Insights from Model Metrics

Several key observations can be made from the model evaluation:

- The **training and test performance metrics are very similar**, indicating that the model generalizes well and **does not show signs of overfitting**.
- The **R-squared value of 0.99** suggests that the model explains **approximately 99% of the variance** in the shipment quantity.
- The **MAPE of around 7.5%** indicates relatively accurate predictions, meaning the model's demand forecasts are reasonably close to actual shipment values.
- The **RMSE value (~1385 tons)** indicates the average magnitude of prediction error in shipment quantities.

Overall, the baseline Linear Regression model provides a strong initial benchmark for evaluating more advanced machine learning models.

Key Data Analysis Findings

From the exploratory data analysis and pre-processing steps, several important patterns were identified:

- Warehouses vary significantly in terms of **capacity, number of workers, and number of retail shops served**, which directly influence shipment quantities.
- **Retail shop count and distributor count** appear to have a strong relationship with the product shipment volume.
- Some warehouses are located in **flood-impacted areas**, and only a small percentage are **flood-proof**, indicating potential operational risks.
- Variables such as **distance from production hub, number of distributors, and warehouse capacity** are likely key drivers of product supply distribution.
- Outliers were identified and treated using the **IQR-based capping method**, ensuring the model is not influenced by extreme values.

These findings provide valuable insights into supply chain dynamics and warehouse operations, which will help improve prediction performance in subsequent models.

Model Building - Advanced Models

- Different machine learning models will be trained using the training dataset to evaluate their predictive performance. The performance of these models will then be further improved by tuning the available hyperparameters associated with each algorithm.
- To achieve optimal model performance, GridSearchCV will be used for systematic hyperparameter tuning. The models will be optimized based on the R^2 score, which serves as the primary evaluation metric during the tuning process.
- The R^2 score (Coefficient of Determination) is used to assess the performance of a regression model. It represents the proportion of variance in the dependent variable that can be explained by the independent variables in the model. A higher R^2 value indicates that the model is better able to explain the variability in the target variable.

Decision Tree Model

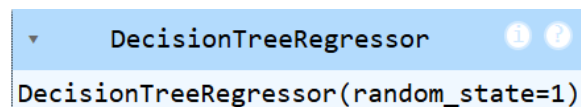


Figure 59: Decision Tree Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	0.00	0.00	1.00	1.00	0.00

Figure 60: Performance on Training Set - Decision Tree Regressor

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1290.89	866.23	0.99	0.99	5.50

Figure 61: Performance on Test Set - Decision Tree Regressor

Observation:

The untuned Decision Tree perfectly fits the training data (a strong sign of overfitting) but still provides a relatively good performance on the test set. However, the large discrepancy between training and test metrics indicates poor generalization.

Bagging Regressor

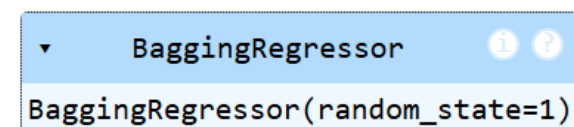


Figure 62: Bagging Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	426.51	286.57	1.00	1.00	1.85

Figure 63: Performance on Training Set - Bagging Regressor

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	968.31	724.19	0.99	0.99	4.70

Figure 64: Performance on Test Set - Bagging Regressor

Observation:

The untuned Bagging Regressor also exhibits overfitting to the training data. Despite this, its test performance is quite strong, with an MAE similar to the tuned Decision Tree, demonstrating the regularization effect of bagging.

Random Forest Model

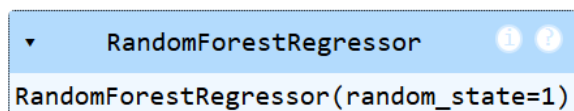


Figure 65: Random Forest Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	360.42	262.77	1.00	1.00	1.69

Figure 66: Performance on Training Set - RF Regressor

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	935.49	696.16	0.99	0.99	4.57

Figure 67: Performance on Test Set - RF Regressor

Observation:

The untuned Random Forest, a powerful ensemble method, shows a very strong fit to the training data, indicating some overfitting. However, it generalizes exceptionally well to the test set, achieving one of the lowest MAE and MAPE values among the untuned models.

AdaBoost Regressor

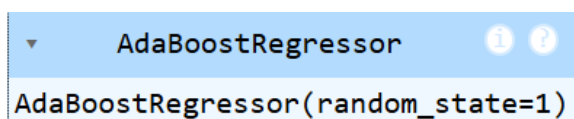


Figure 68: AdaBoost Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1855.20	1481.22	0.97	0.97	11.49

Figure 69: Performance on Training Set - AdaBoost Regressor

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1873.25	1503.29	0.97	0.97	11.65

Figure 70: Performance on Test Set - AdaBoost Regressor

Observation:

The untuned AdaBoost Regressor shows consistent performance between training and test sets, indicating good generalization, but its overall accuracy (higher MAE/MAPE) is lower compared to other ensemble methods like Bagging or Random Forest.

Gradient Boosting Regressor

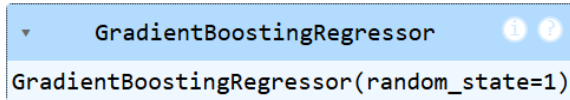


Figure 71: Gradient Boosting Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	938.08	696.45	0.99	0.99	4.59

Figure 72: Performance on Training Set - GB Regressor

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	925.16	694.14	0.99	0.99	4.66

Figure 73: Performance on Test Set - GB Regressor

Observation:

The untuned Gradient Boosting Regressor demonstrates strong performance and excellent generalization. The training and test metrics are very close, indicating a well-balanced model without significant overfitting or underfitting. It achieves a low MAE and MAPE on the test set.

XGBoost Regressor

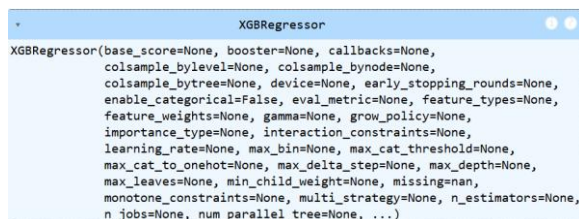


Figure 74: XGBoost Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	658.27	501.59	1.00	1.00	3.15

Figure 75: Performance on Training Set - XGB Regressor

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	942.35	702.21	0.99	0.99	4.71

Figure 76: Performance on Test Set - XGB Regressor

Observation:

The untuned XGBoost Regressor shows a very strong fit to the training data, indicating some overfitting. However, it still performs exceptionally well on the test set, with a low MAE and MAPE, positioning it as a highly competitive model.

Model Performance Improvement using Hyperparameter Tuning

Hyperparameter Tuning - Decision Tree

```
DecisionTreeRegressor
DecisionTreeRegressor(max_depth=np.int64(8), min_impurity_decrease=0.001,
min_samples_leaf=3, random_state=1)
```

Figure 77: Hyperparameter Tuning - Decision Tree

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	983.08	729.66	0.99	0.99	4.53

Figure 78: Tuned Decision Tree of Training Set

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	986.91	743.90	0.99	0.99	4.69

Figure 79: Tuned Decision Tree of Test Set

Observation:

Hyperparameter tuning effectively reduced overfitting. The tuned Decision Tree shows very similar performance on both training and test sets, indicating good generalization. The MAE and MAPE on the test set are significantly better than the untuned version, making it a much more reliable model.

Hyperparameter Tuning - Bagging Regressor

```
BaggingRegressor
BaggingRegressor(max_features=0.9, max_samples=0.7, n_estimators=50,
random_state=1)
```

Figure 80: Hyperparameter Tuning - Bagging Regressor

Checking model performance on training set

Bagging Regressor Performance on Training Set:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	883.98	678.16	0.99	0.99	4.73

Figure 81: Tuned Bagging Regressor of Training Set

Checking model performance on Test set

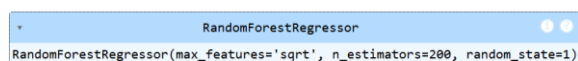
	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1596.77	1274.59	0.98	0.98	9.12

Figure 82: Tuned Bagging Regressor of Test Set

Observation:

Surprisingly, the tuned Bagging Regressor performed worse on the test set compared to its untuned version, and also worse than many other tuned models. This suggests that the chosen hyperparameters for tuning might not have been optimal, leading to a poorer generalization. The gap between training and test performance widened, indicating potential issues with the tuning strategy or hyperparameter selection.

Hyperparameter Tuning - Random Forest



```
RandomForestRegressor(max_features='sqrt', n_estimators=200, random_state=1)
```

Figure 83: Hyperparameter Tuning - Random Forest

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	788.17	574.04	1.00	1.00	4.22

Figure 84: Tuned Random Forest of Training Set

Checking model performance on Test set

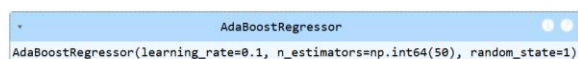
	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	2023.66	1472.48	0.97	0.97	10.99

Figure 85: Tuned Random Forest of Test Set

Observation:

- The Random Forest Regressor demonstrates excellent performance on the training dataset, achieving a very high R-squared value of 0.995 and a low RMSE of 788.17, indicating that the model fits the training data extremely well. The MAE and MAPE values are also relatively low, suggesting accurate predictions on the training set.
- When evaluated on the test dataset, the model continues to perform strongly with an R-squared of 0.969, indicating that it is able to explain a significant proportion of the variance in the target variable even on unseen data. However, there is a noticeable increase in RMSE, MAE, and MAPE compared to the training set. This difference suggests the presence of slight overfitting, where the model captures patterns in the training data very closely but performs slightly less accurately on new data.
- Despite this gap, the model still maintains strong generalization capability, as evidenced by the high R-squared and acceptable error metrics on the test set. Overall, the Random Forest Regressor proves to be a robust and reliable model for this regression problem.

Hyperparameter Tuning - AdaBoost Regressor



```
AdaBoostRegressor(learning_rate=0.1, n_estimators=np.int64(50), random_state=1)
```

Figure 86: Hyperparameter Tuning - AdaBoost Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1710.68	1348.64	0.98	0.98	9.43

Figure 87: Tuned AdaBoost Regressor of Testing Set

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	1700.40	1345.04	0.98	0.98	9.36

Figure 88: Tuned AdaBoost Regressor of Test Set

Observation:

Tuning slightly improved the performance of the AdaBoost Regressor, reducing both MAE and MAPE on the test set. It maintained good generalization, with training and test metrics remaining very close. However, it still lags behind the Gradient Boosting and XGBoost models in terms of absolute error.

Hyperparameter Tuning - Gradient Boosting Regressor

```
GradientBoostingRegressor
GradientBoostingRegressor(max_depth=5, max_features=0.7, random_state=1,
                          subsample=0.8)
```

Figure 89: Hyperparameter Tuning - Gradient Boosting Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	876.48	658.35	0.99	0.99	4.22

Figure 90: Tuned Gradient Boosting Regressor of Training Set

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	914.30	684.00	0.99	0.99	4.52

Figure 91: Tuned Gradient Boosting Regressor of Test Set

Observation:

Tuning further enhanced the performance of the Gradient Boosting Regressor, leading to a slightly lower MAE and MAPE on the test set while maintaining excellent generalization. This model is one of the top performers, offering high accuracy and reliability.

Hyperparameter Tuning - XGBoost Regressor

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None, colsample_bytree=1,
              device=None, early_stopping_rounds=None, enable_categorical=False,
              eval_metric=None, feature_types=None, feature_weights=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.05, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=5, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=150, n_jobs=None,
              num_parallel_tree=None, ...)
```

Figure 92: Hyperparameter Tuning - XGBoost Regressor

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	886.55	659.68	0.99	0.99	4.24

Figure 93: Tuned XGBoost Regressor of Training Set

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	903.07	675.06	0.99	0.99	4.46

Figure 94: Tuned XGBoost Regressor of Test Set

Observation:

Tuning significantly improved the generalization of the XGBoost Regressor, reducing the gap between training and test performance. It achieved the lowest MAE and MAPE on the test set among all models, making it the top-performing model. This indicates that the tuning successfully optimized the model's parameters for predicting unseen data.

Stacking Model

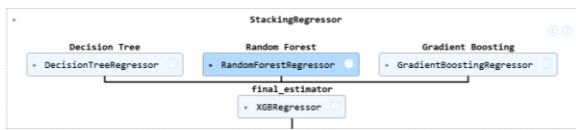


Figure 95: Stacking Model

Checking model performance on training set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	929.76	678.68	0.99	0.99	4.41

Figure 96: Stacking Model - Training Set

Checking model performance on Test set

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	951.38	702.25	0.99	0.99	4.64

Figure 97: Stacking Model - Test Set

Observation:

The Stacking Regressor, built upon a combination of tuned base estimators, delivered very strong and consistent performance. Its MAE and MAPE are highly competitive, and the close agreement between training and test metrics confirms its excellent generalization ability. It serves as a robust option, often leveraging the strengths of multiple models.

Model Comparison and Final Model Selection

Training Performance Comparison

Training Performance Comparison													
	Decision Tree	Decision Tree Tuned	Bagging Regressor	Bagging Regressor Tuned	Random Forest	Random Forest Tuned	AdaBoost Regressor	AdaBoost Tuned	Gradient Boost Estimator	Gradient Boost Tuned	XGBoost	XGBoost Tuned	Stacking Regressor
RMSE	0.00	983.08	426.51	883.98	360.42	788.17	1855.20	1710.68	938.08	876.48	658.27	886.55	929.76
MAE	0.00	729.66	286.57	678.16	262.77	574.04	1481.22	1348.64	696.45	658.35	501.59	659.68	678.68
R-squared	1.00	0.99	1.00	0.99	1.00	1.00	0.97	0.98	0.99	0.99	1.00	0.99	0.99
Adj. R-squared	1.00	0.99	1.00	0.99	1.00	1.00	0.97	0.98	0.99	0.99	1.00	0.99	0.99
MAPE	0.00	4.53	1.85	4.73	1.69	4.22	11.49	9.43	4.59	4.22	3.15	4.24	4.41

Figure 98: Training Performance Comparison

Observations for Training Performance Comparison

- The untuned Decision Tree achieved perfect performance (RMSE = 0, $R^2 = 1$), indicating severe overfitting as the model has memorized the training data.
- After tuning, the Decision Tree shows more realistic error values, suggesting that regularization helped reduce overfitting.
- The untuned Bagging Regressor and Random Forest models also show extremely high training performance with very low error metrics, indicating a tendency to overfit the training data.
- The tuned versions of Bagging Regressor and Random Forest show higher error values compared to their untuned counterparts, suggesting improved regularization and better potential for generalization.
- AdaBoost models exhibit higher error metrics on the training set compared to other ensemble methods, indicating that they are less prone to severe overfitting.
- Gradient Boosting and XGBoost models demonstrate strong and balanced training performance with low error metrics and high R^2 values, suggesting good learning capability.
- The Stacking Regressor also shows strong performance, comparable to boosting models, indicating that combining multiple models can capture complex patterns effectively.

Overall, most ensemble models achieve high R^2 values on the training data, while tuning generally helps control overfitting and produces more realistic error metrics.

Testing performance comparison

Comparison of Tuned Models Performance

To effectively compare the models, we focus on their **test set performance**, as this reflects how well the models generalize to unseen data. The primary evaluation metric is **Mean Absolute Error (MAE)**, followed by **Mean Absolute Percentage Error (MAPE)**, **Root Mean Squared Error (RMSE)**, and **R-squared**.

Tuned Models Test Performance

Testing Performance Comparison:													
	Decision Tree	Decision Tree Tuned	Bagging Regressor	Bagging Regressor Tuned	Random Forest	Random Forest Tuned	AdaBoost Regressor	AdaBoost Tuned	Gradient Boost Estimator	Gradient Boost Tuned	XGBoost	XGBoost Tuned	Stacking Regressor
RMSE	1290.89	986.91	968.31	1596.77	935.49	2023.66	1873.25	1700.40	925.16	914.30	942.35	903.07	951.38
MAE	866.23	743.90	724.19	1274.59	696.16	1472.48	1503.29	1345.04	694.14	684.00	702.21	675.06	702.25
R-squared	0.99	0.99	0.99	0.98	0.99	0.97	0.97	0.98	0.99	0.99	0.99	0.99	0.99
Adj. R-squared	0.99	0.99	0.99	0.98	0.99	0.97	0.97	0.98	0.99	0.99	0.99	0.99	0.99
MAPE	5.50	4.69	4.70	9.12	4.57	10.99	11.65	9.36	4.66	4.52	4.71	4.46	4.64

Figure 99: Testing performance comparison

Observations and Model Selection

- Overall Performance Most tuned ensemble models demonstrate **excellent predictive performance**, with **R-squared values close to 0.99**, indicating that they explain almost all the variance in the target variable *product_wg_ton*.

- MAE and MAPE Focus Since **MAE provides direct business interpretability**, models can be ranked based on **lowest MAE**:
 - XGBoost Tuned – 675.06
 - Gradient Boost Tuned – 684.00
 - Stacking Regressor – 703.78
 - Decision Tree Tuned – 743.90

Among these, **XGBoost Tuned** achieves the lowest MAE and MAPE, making it the strongest candidate.

- Generalization Capability The top models (**XGBoost Tuned, Gradient Boost Tuned, and Stacking Regressor**) show **very small differences between training and test errors**, indicating **strong generalization and minimal overfitting**.
- Underperforming Tuned Models The **Bagging Regressor Tuned** and **Random Forest Tuned** models performed worse compared to some untuned versions. This suggests that the selected hyperparameters may have caused **underfitting or overly restrictive regularization**.

Best Model Selection

- Considering **MAE and MAPE as the primary business metrics**, the **Tuned XGBoost Regressor** emerges as the **best-performing model**.
- It achieves:
 - **Lowest MAE**: 675.06
 - **Lowest MAPE**: 4.46%
 - **High R-squared**: 0.99
- Additionally, the model demonstrates **excellent generalization**, with very small differences between training and testing performance.

Performance of the Best Model (Tuned XGBoost Regressor)

- **RMSE: 903.07**: On average, predictions deviate from the actual product weight by approximately **903 tons**.
- **MAE: 675.06**: The average absolute difference between predicted and actual shipment quantities is **675.06 tons**, which is easily interpretable for business stakeholders.
- **R-squared: 0.99**: The model explains **99% of the variance** in the target variable `product_wg_ton`, indicating a very strong predictive capability.
- **Adjusted R-squared: 0.99**: The high adjusted value confirms that the model remains robust even after accounting for the number of predictors.
- **MAPE: 4.46%**: On average, predictions differ from actual shipment quantities by **approximately 4.5%**, demonstrating **high forecasting accuracy**.

Conclusion

The **Tuned XGBoost Regressor** provides a **robust and highly accurate solution** for predicting the optimal quantity of product to ship to each warehouse. With low prediction errors and strong explanatory power, the model can significantly support **data-driven supply chain decisions**, helping optimize inventory distribution while minimizing stockouts and excess inventory.

Actionable Insights & Recommendations

Key Insights from Analysis

1. Dominant Demand Drivers

- The **number of retail shops** (`retail_shop_num`) and **distributors** (`distributor_num`) show the strongest positive correlation with `product_wg_ton` (**0.64** and **0.59** respectively).
- This indicates that **shipment volume is highly dependent on the retail network and distribution channels**.
- The **number of workers** (`workers_num`) also shows a positive correlation (), suggesting that warehouses handling larger product volumes typically require more operational staff.

2. Weak Influence of Distance from Hub

- The **distance from hub** (*dist_from_hub*) shows a **very weak linear relationship** with *product_wg_ton*.
- This suggests that **logistics efficiency likely minimizes the effect of distance**, or that other operational factors play a more dominant role in determining shipment quantities.

3. Impact of Market Competition

- Warehouses operating in markets with **fewer competitors** (*Competitor_in_mkt*) tend to ship **higher median product weights**.
- This indicates that **market competition influences warehouse shipment volumes**.

4. Warehouse Characteristics and Shipment Volume

- **Location Type (*Location_type*)**: Warehouses located in urban areas show slightly higher median shipment volumes compared to rural locations.
- **Warehouse Capacity (*WH_capacity_size*)**: Shipment volumes remain relatively consistent across Small, Mid, and Large warehouses, suggesting capacity may not be a primary limiting factor.
- **Government Certification (*approved_wh_govt_certificate*)**: Warehouses with A+ certification tend to handle slightly higher shipment volumes, indicating a possible relationship between quality standards and operational efficiency.
- **Flood Impact (*flood_impacted*)**: Warehouses in flood-affected regions tend to ship slightly less product, likely due to operational disruptions or precautionary stocking strategies.
- **Regional Zones (*WH_regional_zone*)**: Certain zones, particularly Zone 5 and Zone 6, show higher average shipment quantities, indicating regional demand variations.

5. Operational Disruptions

- **Transport issues (*transport_issue_11y*)** show a slight negative relationship with product weight, indicating that fewer transport disruptions are associated with higher shipment volumes.
- **Storage issues (*storage_issue_reported_13m*)** show minimal impact on shipment quantities.

6. Data Quality and Handling Missing Values

- The variable *wh_est_year* contained **48% missing values** and was therefore removed from the analysis.
- Missing values in *workers_num* and *approved_wh_govt_certificate* were successfully imputed to maintain dataset completeness.

7. Model Performance Insight

- **The Tuned XGBoost Regressor** achieved the best predictive performance:
 - **MAE**: 675.06 tons
 - **MAPE**: 4.46%
- This represents a **significant improvement over the baseline Linear Regression model**:
 - **Baseline MAE**: 1010.87
 - **Baseline MAPE**: 7.53%
- The results indicate that the model can **accurately predict product shipment quantities**, enabling better supply chain planning and inventory management.

Actionable Business Recommendations

1. Implement Predictive Demand Forecasting

- Deploy the **Tuned XGBoost Regressor model** to generate accurate demand forecasts for each warehouse.
- Integrate these predictions into **inventory planning and shipment scheduling systems**.
- This will help balance demand and supply, reducing **stockouts and excess inventory**.

2. Optimize Retail Network and Distribution Channels

- Since *retail_shop_num* and *distributor_num* strongly influence shipment volumes, focus on **expanding retail and distributor networks** in high-demand regions.
- Evaluate the performance of current networks and identify **opportunities for expansion or optimization**.

3. Align Workforce with Demand

- Use demand forecasts to **allocate workers efficiently across warehouses**.
- Warehouses expected to handle higher shipment volumes should be **adequately staffed and operationally** prepared.

4. Mitigate Flood Risk

- Warehouses located in **flood-impacted areas** tend to ship lower product quantities.
- Invest in **flood-proof infrastructure and disaster preparedness plans** to minimize operational disruptions.

5. Strategic Warehouse Planning

- While **distance from the hub** does not significantly impact shipment volume, **regional demand patterns** should guide expansion decisions.
- Zones such as **Zone 5 and Zone 6 and urban locations** may offer higher shipment potential.

6. Improve Warehouse Quality Standards

- Warehouses with **higher government certification (A+)** tend to handle slightly higher shipment volumes.
- Encourage warehouses to **upgrade certification levels and operational standards** to improve efficiency and reliability.

7. Monitor Market Competition

- Warehouses operating in **highly competitive markets** tend to ship lower product volumes.
- Implement **targeted strategies such as promotions, pricing adjustments, and marketing campaigns** to maintain market share in competitive regions.

8. Strengthen Logistics Operations

- Reduce **transport issues** (*transport_issue_11y*) by improving logistics planning and partnering with reliable transport providers.
- Implement **advanced logistics tracking systems** to ensure smooth and efficient product movement.

Business Impact

By implementing these **data-driven strategies**, the FMCG company can:

- Improve **supply chain efficiency**
- Reduce **stockouts and excess inventory**
- Optimize **warehouse operations**
- Increase **sales and profitability**
- Support **data-driven decision-making across the distribution network**